

UNIT 1

1. Define an Array. What is the difference between a one-dimensional and two-dimensional array?

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**, and each element can be accessed using a common name with an **index (subscript)**.

Arrays are used to store multiple values under a single variable name, which simplifies program structure and allows efficient data access using loops.

Difference between One-Dimensional and Two-Dimensional Arrays

Aspect	One-Dimensional Array (1D)	Two-Dimensional Array (2D)
Definition	Stores elements in a single linear list	Stores elements in rows and columns (tabular form)
Declaration	<code>int a[5];</code>	<code>int a[3][4];</code>
Indexing	Uses one index (e.g., <code>a[2]</code>)	Uses two indices (e.g., <code>a[1][2]</code>)
Memory Representation	Linear and contiguous	Stored row-wise in contiguous memory
Typical Use	List of values (marks, salaries)	Tables, matrices, grids

Examples

- **1D Array:** `int marks[5] = {10, 20, 30, 40, 50};`

2D Array:

```
int matrix[2][2] = {{1, 2}, {3, 4}};
```

Conclusion:

A one-dimensional array represents a simple list, while a two-dimensional array represents data in matrix or table form, suitable for more complex data organization.

2. Write and explain an algorithm to insert and delete elements in a one-dimensional array

(a) Algorithm to Insert an Element in a One-Dimensional Array

Idea:

To insert an element at a given position, elements from that position onwards are shifted **one position to the right**, then the new element is placed at the desired index.

Algorithm: INSERT(A, n, pos, item)

Algorithm INSERT(A, n, pos, item)

// Input : Array A of size n, position pos, element item

// Output: Array after insertion

1. If $\text{pos} < 0$ or $\text{pos} > n$
Print "Invalid Position" and Stop
2. For $i \leftarrow n$ down to $\text{pos} + 1$
 $A[i] \leftarrow A[i - 1]$
3. $A[\text{pos}] \leftarrow \text{item}$
4. $n \leftarrow n + 1$
5. Stop

Explanation:

- Step 2 shifts elements to create space.
- Step 3 inserts the new element.
- Time Complexity: $O(n)$ due to shifting.

(b) Algorithm to Delete an Element from a One-Dimensional Array

Idea:

To delete an element from a given position, elements after that position are shifted **one position to the left**.

Algorithm: DELETE(A, n, pos)

Algorithm DELETE(A, n, pos)

// Input : Array A of size n, position pos

// Output: Array after deletion

1. If $\text{pos} < 0$ or $\text{pos} \geq n$
Print "Invalid Position" and Stop
2. For $i \leftarrow \text{pos}$ to $n - 2$
 $A[i] \leftarrow A[i + 1]$
3. $n \leftarrow n - 1$

4. Stop

Explanation:

- Step 2 removes the element by overwriting it.
- Array size is reduced after deletion.
- Time Complexity: $O(n)$ due to shifting.

Conclusion

Insertion and deletion in a one-dimensional array require shifting elements, making both operations **time-consuming** ($O(n)$), which is a key limitation of arrays.

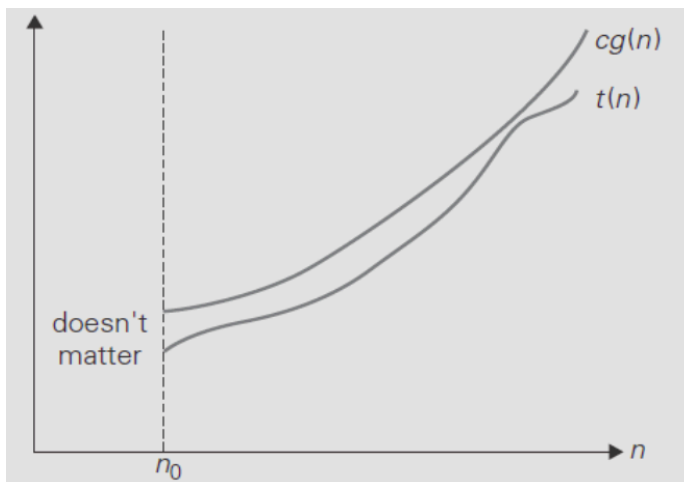
3. With the help of neat diagram explain different asymptotic notations

Asymptotic notations are used to describe the **growth rate of an algorithm's time or space complexity** as input size n becomes large. They ignore constants and lower-order terms and focus on scalability.

1. Big-O Notation – $O(g(n))$

- Represents the **upper bound** of an algorithm's running time.
- Used for **worst-case analysis**.
- Definition:
An algorithm with time $t(n)$ is $O(g(n))$ if there exist constants c and n_0 such that
 $t(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Diagram idea:



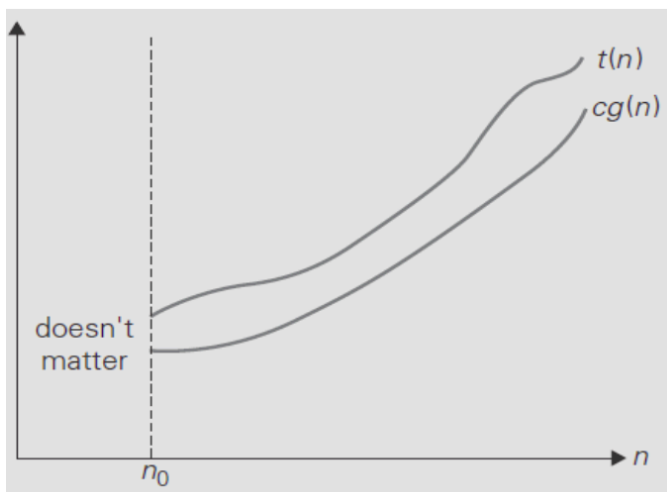
Example:

$$3n^2 + 5n + 7 \in O(n^2)$$

2. Big- Ω Notation – $\Omega(g(n))$

- Represents the **lower bound** of an algorithm's running time.
- Used for **best-case analysis**.
- Definition:
 $t(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

Diagram idea:



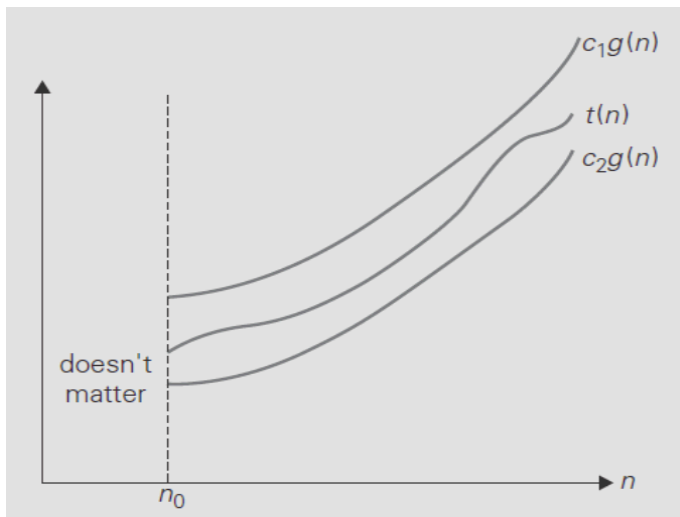
Example:

$$3n^2 + 5n + 7 \in \Omega(n^2)$$

3. Big- Θ Notation – $\Theta(g(n))$

- Represents the **tight bound** of an algorithm.
- Used for **average-case or exact growth rate**.
- Definition:
 $c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq n_0$.

Diagram idea:



Example:

$$3n^2 + 5n + 7 \in \Theta(n^2)$$

Summary (Exam-friendly)

- $O(g(n)) \rightarrow$ Upper bound $\rightarrow$ Worst case
- $\Omega(g(n)) \rightarrow$ Lower bound $\rightarrow$ Best case
- $\Theta(g(n)) \rightarrow$ Tight bound $\rightarrow$ Exact growth

4. Differentiate between Best-case, Worst-case, and Average-case Time Complexities

Time complexity describes how an algorithm's running time varies with **different inputs of the same size**. Based on input conditions, it is classified into **best-case**, **worst-case**, and **average-case**.

1. Best-Case Time Complexity

- Represents the **minimum time** taken by an algorithm.
- Occurs for the **most favorable input**.

- Denoted as Ω (Big-Omega).
- Shows how fast an algorithm can run at best.

Example (Linear Search):

If the search key is found at the **first position**, only one comparison is needed.

Best case = $\Omega(1)$

2. Worst-Case Time Complexity

- Represents the **maximum time** taken by an algorithm.
- Occurs for the **least favorable input**.
- Denoted as O (Big-O).
- Most commonly used for algorithm analysis.

Example (Linear Search):

If the element is found at the **last position** or not found at all, **n** comparisons are needed.

Worst case = $O(n)$

3. Average-Case Time Complexity

- Represents the **expected time** taken over all possible inputs.
- Lies **between best case and worst case**.
- Denoted as Θ (Big-Theta).
- Requires assumptions about input distribution.

Example (Linear Search):

If the element is equally likely to be at any position, average comparisons $\approx n/2$.

Average case = $\Theta(n)$

Comparison Table

Case	Meaning	Notation	Linear Search Example
Best Case	Minimum time	Ω	Element at first position (1 comparison)
Worst Case	Maximum time	O	Element at last / not present (n comparisons)
Average Case	Expected time	Θ	Element in middle ($\sim n/2$ comparisons)

5. List and Explain Different Methods of Specifying Algorithms

An **algorithm** can be specified in different ways to clearly describe the steps involved in solving a problem. The commonly used methods of specifying algorithms are the following:

1. Natural Language

- Algorithms are described using **plain English sentences**.
- Easy to understand for beginners.
- However, it may become **ambiguous and unclear** for complex problems.

Example:

“Read two numbers, add them, and display the result.”

2. Pseudocode

- A **semi-formal representation** combining natural language with programming constructs.
- Uses keywords like **if**, **while**, **for**, and assignment operator $\leftarrow$.
- Easy to convert into actual program code.
- Most **commonly used method** in algorithm design.

Example:

```
read a, b
c ← a + b
print c
```

3. Flowchart

- A **graphical representation** of an algorithm.
- Uses standard symbols like:
 - Oval → Start/Stop
 - Parallelogram → Input/Output
 - Rectangle → Processing
 - Diamond → Decision
- Makes logic easy to visualize but becomes complex for large algorithms.

6. Define Structure in C. How is it different from an array?

Structure in C

A **structure** in C is a **user-defined data type** that allows grouping of **different data types** under a single name.

Each element inside a structure is called a **member**, and members can be of different types such as **int**, **float**, **char**, arrays, or even other structures.

Example:

```
struct student {  
    int rollno;  
    char name[30];  
    float marks;  
};
```

Difference between Structure and Array

Aspect	Structure (struct)	Array
Data type stored	Can store different data types	Stores same data type only
Nature	User-defined data type	Derived data type
Access method	Members accessed using dot (.) operator	Elements accessed using index
Memory allocation	Memory allocated for each member separately	Memory allocated for all elements of same type
Use case	Represents records (student, employee)	Represents list of values
Data relationship	Members can be unrelated	Elements are homogeneous
Size	Size = sum of sizes of all members	Size = size of one element × number of elements
Assignment	Structure variables can be assigned directly	Arrays cannot be assigned directly
Function passing	Can be passed by value or reference	Passed only as address (pointer)
Flexibility	More flexible and meaningful	Less flexible

7. Write a Structure program to create a student structure. Display highest scored student details and average marks of all the students.

```
#include <stdio.h>

struct student {
    int regno;
    char name[30];
    float marks;
};

int main() {
    struct student s[50];
    int n, i, maxIndex = 0;
    float sum = 0, avg;

    printf("Enter number of students: ");
    scanf("%d", &n);

    // Input student details
    for (i = 0; i < n; i++) {
        printf("\nEnter Register Number: ");
        scanf("%d", &s[i].regno);
        printf("Enter Name: ");
        scanf("%s", s[i].name);
        printf("Enter Marks: ");
        scanf("%f", &s[i].marks);

        sum += s[i].marks;

        if (s[i].marks > s[maxIndex].marks) {
            maxIndex = i;
        }
    }

    // Calculate average
    avg = sum / n;

    // Display highest scored student
    printf("\n--- Highest Scored Student ---\n");
```

```
printf("Register Number: %d\n", s[maxIndex].regno);
printf("Name: %s\n", s[maxIndex].name);
printf("Marks: %.2f\n", s[maxIndex].marks);

// Display average marks
printf("\nAverage Marks of all students: %.2f\n", avg);

return 0;
}
```

Explanation (brief):

- A **student** structure is defined with register number, name, and marks.
- An array of structures stores details of multiple students.
- While reading input, total marks and highest marks are tracked.
- Average marks are calculated and highest scored student details are displayed.

8. List and Explain different types of Data Structures with example

A **data structure** is a way of organizing and storing data so that it can be accessed and processed efficiently. Data structures are broadly classified as **Primitive** and **Non-Primitive** data structures.

1. Primitive Data Structures

- These are **basic data types** directly supported by the programming language.
- They store **single values**.
- Examples include: **int**, **float**, **char**, **double**, **pointer**.

Example:

```
int count;
float marks;
```

2. Non-Primitive Data Structures

- These are **derived from primitive data types**.
- They can store **multiple values** and represent relationships among data.

- Non-primitive data structures are further classified into **Linear** and **Non-Linear**.
-

(a) Linear Data Structures

- Data elements are arranged in a **sequential order**.
- Each element is connected to its previous and next element.

Examples:

- **Array** – Stores elements of same data type
`int a[5];`
 - **Stack** – Follows **LIFO (Last In First Out)**
Example: Stack of plates
 - **Queue** – Follows **FIFO (First In First Out)**
Example: People standing in a queue
 - **Linked List** – Collection of nodes linked using pointers
-

(b) Non-Linear Data Structures

- Data elements are arranged in a **hierarchical or graph form**.
- No sequential ordering.

Examples:

- **Tree** – Represents hierarchical relationships
Example: File system
 - **Graph** – Represents networks of nodes and edges
Example: Road maps, computer networks
-

Summary

- **Primitive** → Stores single values
- **Non-Primitive** → Stores multiple related values
 - **Linear** → Array, Stack, Queue, Linked List
 - **Non-Linear** → Tree, Graph

9. Briefly explain how algorithm efficiency is analyzed

Algorithm efficiency analysis is the process of evaluating how effectively an algorithm uses **time and memory** as the input size increases. It helps in comparing algorithms and selecting the most efficient one.

Steps in Analyzing Algorithm Efficiency

1. **Identify the Input Size (n)**
 - Determine the parameter that represents the size of input (e.g., number of elements in an array).
 2. **Identify the Basic Operation**
 - Select the most frequently executed operation (comparison, addition, multiplication, etc.).
 3. **Count the Number of Basic Operations**
 - Express how many times the basic operation executes as a function of n .
 4. **Determine Time and Space Efficiency**
 - **Time efficiency:** Measures how fast the algorithm runs.
 - **Space efficiency:** Measures extra memory used by the algorithm.
 5. **Use Asymptotic Notations**
 - Express efficiency using O (Big-O), Ω (Big-Omega), and Θ (Big-Theta) to describe growth rate.
 6. **Consider Case Analysis**
 - Analyze **best-case**, **worst-case**, and **average-case** scenarios.
-

This method focuses on the **order of growth** rather than exact execution time, making it machine-independent and suitable for comparing algorithms.

10. Explain how a two-dimensional array is represented in memory, and describe the method used to calculate the address of an element in a two-dimensional array

Memory Representation of a Two-Dimensional Array

A two-dimensional array is stored in contiguous memory locations in **row-major order** in C.

This means **all elements of the first row are stored first**, followed by elements of the second row, and so on.

Example:

`int a[2][3] = { {1, 2, 3}, {4, 5, 6} };`

Memory Layout:

`a[0][0] → 1`

`a[0][1] → 2`

`a[0][2] → 3`

`a[1][0] → 4`

`a[1][1] → 5`

`a[1][2] → 6`

All elements are stored **sequentially row by row**.

Address Calculation of an Element

The address of an element `a[i][j]` in a 2D array using **row-major order** is calculated using the formula:

$\text{Address}(a[i][j]) = \text{Base Address} + [(i \times \text{Number of Columns}) + j] \times \text{Size of Data Type}$

Explanation of Terms

- **Base Address** → Address of `a[0][0]`
- **i** → Row index
- **j** → Column index
- **Number of Columns** → Total columns in the array
- **Size of Data Type** → Size of each element (e.g., 4 bytes for `int`)

Example Calculation

For `int a[2][3]`, find the address of `a[1][2]` if base address is 1000 and size of `int` is 4 bytes:

```
[
\text{Address} = 1000 + [(1 \times 3) + 2] \times 4
]
[
= 1000 + (5 \times 4) = 1020
]
```

Thus, a two-dimensional array is stored row-wise in memory, and element addresses are computed using row-major order addressing.

11. Define Abstract Data Type (ADT). Explain with example

An Abstract Data Type (ADT) is a **logical specification** of a data type that defines **what data** is stored and **what operations** can be performed, **without specifying how they are implemented**.

It focuses on **behavior (operations)** rather than **implementation details (data structure)**.

Components of an ADT

1. **Value Definition** – Specifies the type of data and valid values.
 2. **Operation Definition** – Specifies the operations that can be performed on the data.
-

Example: ADT – RATIONAL

A RATIONAL number represents a fraction of two integers.

Value Definition:

- A rational number consists of two integers (**numerator, denominator**)
- Denominator $\neq 0$

Operations Defined:

- **makerational(a, b)** $\rightarrow$ creates a rational number **a/b**
- **add(r1, r2)** $\rightarrow$ adds two rational numbers
- **mult(r1, r2)** $\rightarrow$ multiplies two rational numbers
- **equal(r1, r2)** $\rightarrow$ checks equality of two rational numbers

Equality Check Example:

Two rational numbers **a/b** and **c/d** are equal if

[
 $a \times d = b \times c$
]

Key Point

The ADT specifies **what operations are allowed**, not **how they are coded** (array, structure, etc.). The implementation can vary without changing the ADT.

12. Write an ADT for Rational Numbers

An ADT for Rational Numbers defines the data type and operations on rational numbers **independently of implementation**. A rational number is represented as a pair of integers (**numerator**, **denominator**) where the denominator is non-zero.

ADT Specification: RATIONAL

Value Definition

abstract typedef <integer, integer> RATIONAL
condition RATIONAL[1] $\neq$ 0

- **RATIONAL[0]** $\rightarrow$ numerator
 - **RATIONAL[1]** $\rightarrow$ denominator (must not be 0)
-

Operation Definitions

abstract RATIONAL makerational(a, b)
int a, b
precondition b $\neq$ 0
postcondition makerational[0] = a
postcondition makerational[1] = b

abstract RATIONAL add(r1, r2)
RATIONAL r1, r2
postcondition add[0] = r1[0] * r2[1] + r2[0] * r1[1]
postcondition add[1] = r1[1] * r2[1]

abstract RATIONAL mult(r1, r2)
RATIONAL r1, r2
postcondition mult[0] = r1[0] * r2[0]
postcondition mult[1] = r1[1] * r2[1]

abstract boolean equal(r1, r2)
RATIONAL r1, r2
postcondition equal = (r1[0] * r2[1] == r2[0] * r1[1])

Note:

This ADT specifies **what** operations do, not **how** they are implemented. Different implementations (e.g., structures, classes) can realize this ADT without changing its specification.

13. Write Stack and Queue as an ADT

Stack as an ADT

A **Stack ADT** is a linear data structure that follows the **LIFO (Last In First Out)** principle. Only the **top** element is accessible.

ADT Specification: STACK

Value Definition

abstract typedef <eltype> STACK

Operation Definitions

abstract boolean empty(S)
STACK S
postcondition empty = (len(S) == 0)

abstract push(S, x)
STACK S
eltype x
postcondition S = <x> + S'

abstract eltype pop(S)
STACK S
precondition empty(S) = FALSE
postcondition pop = first(S')
S = sub(S', 1, len(S') - 1)

Queue as an ADT

A **Queue ADT** is a linear data structure that follows the **FIFO (First In First Out)** principle. Insertion is done at the **rear**, deletion at the **front**.

ADT Specification: QUEUE

Value Definition

abstract typedef <eltype> QUEUE

Operation Definitions

abstract boolean empty(Q)
QUEUE Q
postcondition empty = (len(Q) == 0)

abstract enqueue(Q, x)
QUEUE Q
eltype x
postcondition Q = Q' + <x>

abstract eltype dequeue(Q)
QUEUE Q
precondition empty(Q) = FALSE
postcondition dequeue = first(Q')
Q = sub(Q', 1, len(Q') - 1)

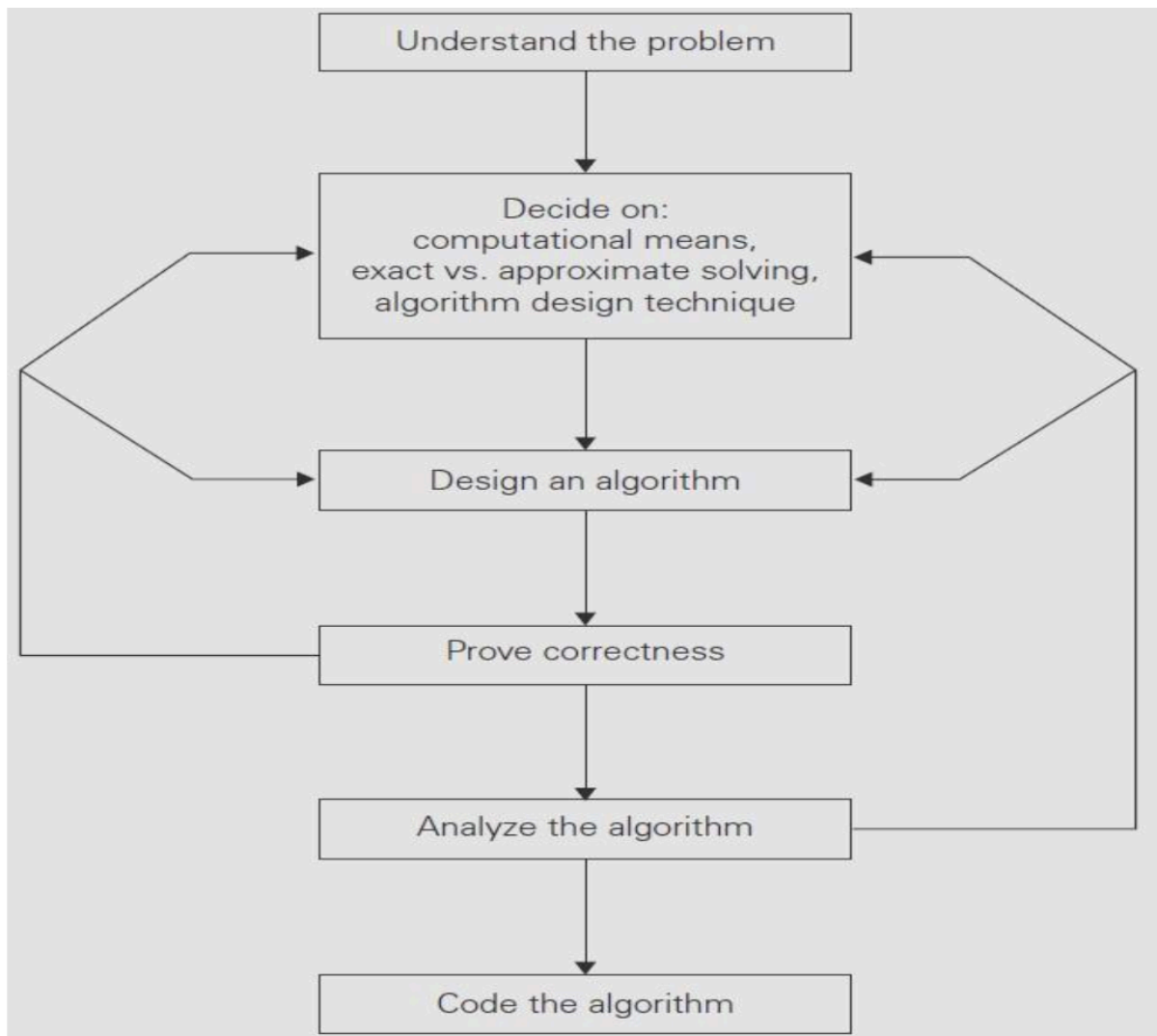
Note:

These ADTs define **what operations are performed** on Stack and Queue, not **how they are implemented** (array or linked list).

14. With a neat diagram explain the Algorithm Development Process

The **algorithm development process** consists of a sequence of well-defined steps followed to design, analyze, and implement an algorithm efficiently. It ensures correctness, efficiency, and clarity in problem solving.

Neat Diagram: Algorithm Development Process



Explanation of Each Step

1. Understanding the Problem

- Clearly understand the problem statement.
- Identify inputs, outputs, and constraints.

2. Decision Making

- Choose the computational model (sequential or parallel).
- Decide between **exact** or **approximate** solution.
- Select suitable **algorithm design technique** (Brute Force, Divide & Conquer, Greedy, etc.).

3. Algorithm Specification

- Describe the algorithm using **natural language**, **pseudocode**, or **flowchart**.
- Pseudocode is most commonly preferred.

4. Proving Correctness

- Ensure the algorithm produces correct output for all valid inputs.

- Often proved using **logical reasoning or mathematical induction**.
 - 5. **Analyzing Efficiency**
 - Analyze **time complexity** and **space complexity**.
 - Use asymptotic notations like O , Ω , Θ .
 - 6. **Coding / Implementation**
 - Convert the algorithm into a program using a programming language.
 - Optimize code without changing algorithm logic.
-

15. Distinguish between Algorithm and Pseudocode

An **algorithm** is a logical, step-by-step method to solve a problem, while **pseudocode** is a structured way of writing that algorithm using programming-like constructs for better clarity and implementation.

Differences between Algorithm and Pseudocode

Aspect	Algorithm	Pseudocode
Definition	A finite sequence of clear steps to solve a problem	A semi-formal representation of an algorithm
Language Style	Written in simple natural language	Uses programming-like statements
Syntax Rules	No strict syntax rules	Follows structured syntax (if, while, for)
Ambiguity	May be ambiguous for complex problems	Less ambiguous and more precise
Implementation	Difficult to convert directly into code	Easy to convert into program code
Usage	Used to explain logic at a high level	Used before actual coding

Example

Algorithm (Addition of two numbers):

1. Read two numbers

2. Add them
3. Display the result

Pseudocode:

```
read a, b
c ← a + b
print c
```

16. Distinguish between Big-Oh, Big-Omega, and Big-Theta Notation

Asymptotic notations are used to describe an algorithm's **growth rate** with respect to input size **n**. The three main notations are **Big-O**, **Big-Ω**, and **Big-Θ**.

Differences

Aspect	Big-O (O)	Big-Omega (Ω)	Big-Theta (Θ)
Meaning	Upper bound of running time	Lower bound of running time	Tight (exact) bound of running time
Case Represented	Worst case	Best case	Average / exact growth
Bound Type	Loose upper bound	Loose lower bound	Tight bound (both upper & lower)
Definition	$t(n) \leq c \cdot g(n)$ for $n \geq n_0$	$t(n) \geq c \cdot g(n)$ for $n \geq n_0$	$c_1 \cdot g(n) \leq t(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$
Usage	Most commonly used in analysis	Shows minimum possible time	Shows precise order of growth

Examples

- $3n^2 + 5n + 7 \in O(n^2)$
 - $3n^2 + 5n + 7 \in \Omega(n^2)$
 - $3n^2 + 5n + 7 \in \Theta(n^2)$
-

Summary:

- $O \rightarrow$ Worst-case upper limit
- $\Omega \rightarrow$ Best-case lower limit
- $\Theta \rightarrow$ Exact growth rate

17.Explain asymptotic notations – Big O, Omega (Ω), and Theta (Θ) – with graphical representation and examples

Asymptotic notations are mathematical tools used to describe the **growth rate of an algorithm's time or space complexity** as input size n becomes large. They help compare algorithms independent of machine and implementation details.

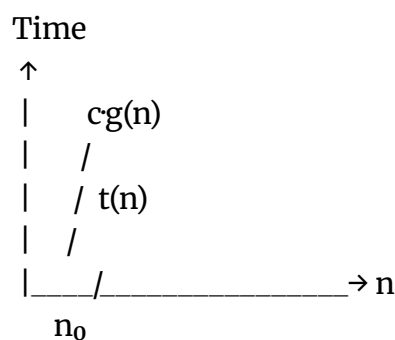
1. Big-O Notation – $O(g(n))$

- Represents the **upper bound** on the running time.
- Used to describe the **worst-case time complexity**.
- Shows the maximum time an algorithm can take.

Definition:

$t(n) \in O(g(n))$ if there exist constants c and n_0 such that $t(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Graphical representation:



Example:

$$3n^2 + 5n + 7 \in O(n^2)$$

2. Big-Omega Notation – $\Omega(g(n))$

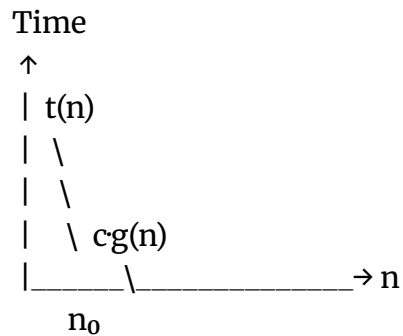
- Represents the **lower bound** on the running time.

- Used to describe the **best-case time complexity**.
- Shows the minimum time required by an algorithm.

Definition:

$t(n) \in \Omega(g(n))$ if there exist constants c and n_0 such that $t(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

Graphical representation:



Example:

$$3n^2 + 5n + 7 \in \Omega(n^2)$$

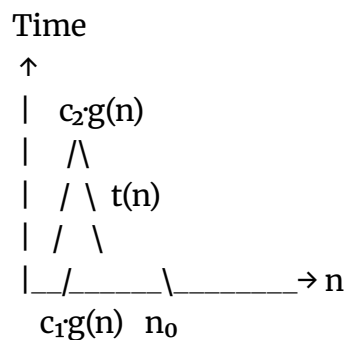
3. Big-Theta Notation – $\Theta(g(n))$

- Represents the **tight bound** on running time.
- Describes the **exact order of growth**.
- Used for **average-case or precise analysis**.

Definition:

$t(n) \in \Theta(g(n))$ if there exist constants c_1 , c_2 , and n_0 such that $c_1 \cdot g(n) \leq t(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

Graphical representation:



Example:

$$3n^2 + 5n + 7 \in \Theta(n^2)$$

Summary

- **Big O (O)** → Upper bound → Worst case
- **Big Ω (Ω)** → Lower bound → Best case
- **Big Θ (Θ)** → Tight bound → Exact growth

18. Explain Best-case, Worst-case and Average-case Time Complexities with graphical representation and examples

An algorithm may take **different amounts of time** for different inputs of the same **size**. Based on this, time complexity is classified into **best-case, worst-case, and average-case**.

1. Best-Case Time Complexity

- Represents the **minimum time** taken by an algorithm.
- Occurs for the **most favorable input**.
- Denoted using Ω (Big-Omega).

Graphical Representation

Time

↑

| Worst case

| /

| /

| /

| Best case

|_____/_____→ n

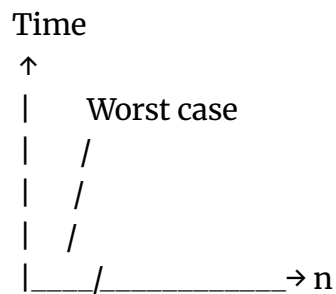
Example (Linear Search)

- If the element is found at the **first position**, only **one comparison** is required.
- Best case = $\Omega(1)$

2. Worst-Case Time Complexity

- Represents the **maximum time** taken by an algorithm.
- Occurs for the **least favorable input**.
- Denoted using **O (Big-O)**.
- Most commonly used for analysis.

Graphical Representation



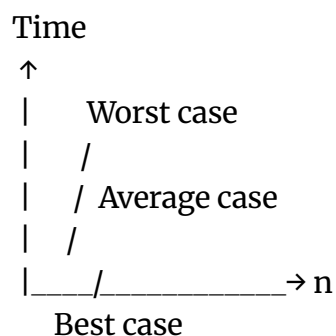
Example (Linear Search)

- If the element is at the **last position** or **not present**, **n** comparisons are needed.
- Worst case = **O(n)**

3. Average-Case Time Complexity

- Represents the **expected running time** over all possible inputs.
- Lies **between best and worst cases**.
- Denoted using **Θ (Big-Theta)**.
- Requires assumptions about input distribution.

Graphical Representation



Example (Linear Search)

- If the element is equally likely to be anywhere,
- Average comparisons $\approx n/2$
- Average case = $\Theta(n)$

Summary Table

Case	Meaning	Notation	Linear Search Example
Best Case	Minimum time	Ω	Element at first position
Worst Case	Maximum time	O	Element at last / not found
Average Case	Expected time	Θ	Element in middle

19. How does structure differ from an array? Write a structure program to explain the concept of nested structure

Difference between Structure and Array

Aspect	Structure	Array
Data Type Stored	Can store different data types	Stores same data type only
Nature	User-defined data type	Derived data type
Access Method	Accessed using dot (.) operator	Accessed using index
Memory	Members may be of different sizes	All elements are of same size
Assignment	Structure variables can be assigned directly	Arrays cannot be assigned directly
Usage	Represents real-world entities	Represents list of similar items

Nested Structure

A **nested structure** is a structure **declared inside another structure**. It is used to logically group related data.

Program: Nested Structure Example

```
#include <stdio.h>

struct Address {
    char street[30];
    char city[20];
    int pincode;
};

struct Student {
    int regno;
    char name[30];
    struct Address addr; // Nested structure
};

int main() {
    struct Student s;

    printf("Enter Register Number: ");
    scanf("%d", &s.regno);
    printf("Enter Name: ");
    scanf("%s", s.name);
    printf("Enter Street: ");
    scanf("%s", s.addr.street);
    printf("Enter City: ");
    scanf("%s", s.addr.city);
    printf("Enter Pincode: ");
    scanf("%d", &s.addr.pincode);

    printf("\n--- Student Details ---\n");
    printf("Reg No: %d\n", s.regno);
    printf("Name: %s\n", s.name);
    printf("Address: %s, %s - %d\n",
           s.addr.street, s.addr.city, s.addr.pincode);

    return 0;
}
```

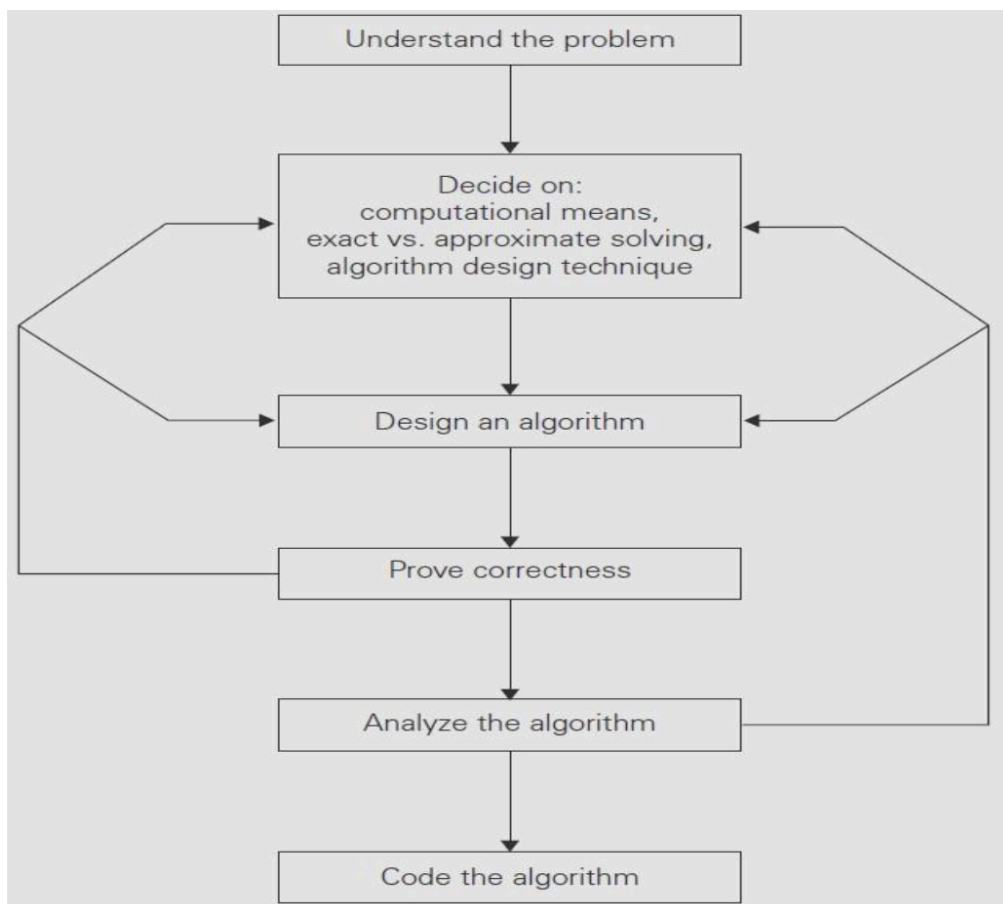
Explanation

- **Address** structure is nested inside **Student**.
- Members of nested structure are accessed using **dot operator** twice (e.g., **s.addr.city**).
- Useful for representing **complex real-world data** like student details with address.

20. With a neat diagram explain the Algorithm Development Process [repeated question]

The algorithm development process is a systematic sequence of steps followed to design, analyze, and implement an algorithm correctly and efficiently.

Neat Diagram: Algorithm Development Process



Explanation of Each Step

1. Understanding the Problem

- Clearly understand the problem statement.
- Identify inputs, outputs, and constraints.

2. Decision Making

- Decide the computational model (sequential/parallel).
- Choose between **exact** or **approximate** solutions.
- Select an appropriate **algorithm design technique** (Brute force, Divide & Conquer, Greedy, etc.).

3. Algorithm Specification

- Write the algorithm using **natural language**, **pseudocode**, or **flowchart**.
- Pseudocode is preferred due to clarity and ease of coding.

4. Proving Correctness

- Ensure the algorithm produces correct results for all valid inputs.
- Often done using logical reasoning or mathematical induction.

5. Efficiency Analysis

- Analyze **time efficiency** and **space efficiency**.
- Use asymptotic notations such as O , Ω , Θ .

6. Coding / Implementation

- Convert the algorithm into a program using a programming language.
- Optimize code without changing the algorithm logic.

UNIT 2

1. Differentiate between Singly, Doubly, and Circular Linked Lists

Linked lists are dynamic data structures where elements (nodes) are connected using pointers. Based on pointer structure and traversal, they are classified as **singly**, **doubly**, and **circular** linked lists.

Aspect	Singly Linked List (SLL)	Doubly Linked List (DLL)	Circular Linked List (CLL)
Node Structure	Data + one pointer (next)	Data + two pointers (left, right)	Data + pointer(s) forming a circle
Traversal	Only forward direction	Forward and backward traversal	Traversal is circular , no NULL end
Last Node Pointer	next = NULL	right = NULL	Points back to first node
Deletion Ease	Deletion needs previous node	Deletion possible with given node pointer	Easier than SLL, but needs care
Memory Usage	Less (one pointer)	More (two pointers)	Same as SLL or DLL (depends on type)
Implementation Complexity	Simple	More complex	Moderately complex

Examples

- **Singly Linked List:**
 $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$
- **Doubly Linked List:**
 $\text{NULL} \leftarrow 10 \rightleftarrows 20 \rightleftarrows 30 \rightarrow \text{NULL}$
- **Circular Linked List:**
 $10 \rightarrow 20 \rightarrow 30 \rightarrow (\text{back to } 10)$

2. What is a linked list? How does it differ from an array?

Linked List

A **linked list** is a **dynamic data structure** consisting of a collection of **nodes**, where each node contains:

- a **data (info)** field, and
- a **pointer (link)** to the next node in the sequence.

The nodes are **not stored in contiguous memory locations**; instead, they are connected through pointers.

Difference between Linked List and Array

Aspect	Linked List	Array
Memory Allocation	Dynamic memory allocation	Static (fixed size) memory allocation
Memory Location	Non-contiguous memory	Contiguous memory locations
Size	Can grow or shrink at runtime	Fixed size once declared
Insertion / Deletion	Easy (no shifting of elements)	Difficult (requires shifting)
Access Method	Sequential access only	Random access using index
Memory Overhead	Extra memory for pointer(s)	No extra pointer memory
Traversal Speed	Slower due to pointer traversal	Faster due to direct indexing

Example:

- Array: `int a[5];`
- Linked List: `10 → 20 → 30 → NULL`

3. Write C implementation code to delete all the occurrences of a given number from the singly linked list

Below is a C program function that deletes all occurrences of a given value `x` from a singly linked list, based strictly on the SLL realization concepts you uploaded.

C Implementation (Singly Linked List)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {  
    int info;  
    struct node *next;  
};
```

```
typedef struct node* NODEPTR;
```

```
NODEPTR list = NULL;
```

```
/* Function to delete all occurrences of x */
```

```
void delete_all(int x)
```

```
{
```

```
    NODEPTR p, q;
```

```
    /* Delete occurrences at the beginning */
```

```
    while (list != NULL && list->info == x)
```

```
    {
```

```
        p = list;
```

```
        list = list->next;
```

```
        free(p);
```

```
    }
```

```
    /* Delete occurrences in the remaining list */
```

```
    q = list;
```

```
    p = (list != NULL) ? list->next : NULL;
```

```
    while (p != NULL)
```

```
    {
```

```
        if (p->info == x)
```

```
        {
```

```
            q->next = p->next;
```

```
            free(p);
```

```
            p = q->next;
```

```
        }
```

```
    else
```

```
    {
```

```
        q = p;
```

```
        p = p->next;
```

```
    }
```

```
}  
}
```

Explanation (Exam-oriented)

- First, all matching nodes at the **beginning of the list** are deleted.
- Then the list is traversed using two pointers:
 - **q** → previous node
 - **p** → current node
- When **p->info == x**, the node is removed by adjusting links and freeing memory.
- Traversal continues until the end of the list.

This method efficiently removes **all occurrences** of the given element from a **singly linked list** without using extra memory.

4. Insert a node into a singly linked list after a given node

To insert a new node **after a given node p** in a **singly linked list**, the links are adjusted so that the new node is placed immediately after **p**.

C Implementation

```
void insert_after(NODEPTR p, int x)
{
    NODEPTR q;

    if (p == NULL)
        printf("Invalid insertion\n");
    else
    {
        q = getnode();    // create new node
        q->info = x;      // assign data
        q->next = p->next; // link new node to next of p
        p->next = q;      // link p to new node
    }
}
```

Explanation

- **p** is the pointer to the **given node**.
- A new node **q** is created and data **x** is stored.
- **q->next** is made to point to the node after **p**.
- **p->next** is updated to point to **q**.
- If **p** is **NULL**, insertion is not possible.

5. Explain the structure of a node in a singly linked list with an example in C

In a **singly linked list**, each element is called a **node**. A node contains **two parts**:

1. an **information (data) field**, and
2. a **link (pointer) field** that stores the address of the **next node** in the list.

Structure of a Node

```
struct node {  
    int info;          // Data field  
    struct node *next; // Link to next node  
};
```

- **info**: Stores the data value.
- **next**: Stores the address of the next node in the list.
- The last node's **next** pointer is set to **NULL**, indicating the end of the list.

Example of a Node in a Singly Linked List

```
struct node {  
    int info;  
    struct node *next;  
};
```

```
struct node *p;
```

```
p = (struct node*)malloc(sizeof(struct node));  
p->info = 10;  
p->next = NULL;
```

Illustration

```
+-----+-----+
| info | next |
+-----+-----+
| 10  | NULL |
+-----+-----+
```

This structure enables **dynamic memory allocation**, efficient insertion, and deletion operations in a singly linked list.

6. What are the advantages and disadvantages of using a singly linked list?

A **singly linked list (SLL)** is a dynamic data structure in which each node contains a data field and a pointer to the next node. It has both benefits and limitations.

Advantages of Singly Linked List

1. **Dynamic size**
 - The list can grow or shrink during program execution.
2. **Efficient insertion and deletion**
 - Elements can be inserted or deleted without shifting other elements.
3. **Better memory utilization than doubly linked list**
 - Requires only one pointer per node.
4. **No contiguous memory required**
 - Nodes can be stored anywhere in memory.

Disadvantages of Singly Linked List

1. **Sequential access only**
 - Elements cannot be accessed directly using index; traversal is required.
2. **No backward traversal**
 - Nodes can be traversed only in forward direction.
3. **Extra memory for pointer**
 - Each node needs extra memory for storing the link.
4. **Deletion requires previous node**
 - Direct deletion of a node is not possible without its predecessor.

7. What is the difference between Singly Linked List and Doubly Linked List?

A singly linked list (SLL) and a doubly linked list (DLL) differ mainly in the number of pointers per node, traversal capability, and ease of deletion.

Differences between Singly Linked List and Doubly Linked List

Aspect	Singly Linked List (SLL)	Doubly Linked List (DLL)
Pointers per Node	One pointer (next)	Two pointers (left , right)
Traversal	Only forward	Forward and backward
Node Structure	Data + next pointer	Data + left & right pointers
Deletion	Requires previous node	Can delete using given node pointer
Memory Requirement	Less memory	More memory due to extra pointer
Implementation	Simple	More complex

Example Representation

- Singly Linked List:
 $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$
- Doubly Linked List:
 $\text{NULL} \leftarrow 10 \rightleftarrows 20 \rightleftarrows 30 \rightarrow \text{NULL}$

8. How is memory allocated for nodes in a linked list?

In a linked list, memory for each node is allocated **dynamically at run time** using **dynamic memory allocation**. Unlike arrays, linked list nodes are **not stored in contiguous memory locations**.

Memory Allocation Method

- Each node is created **when required** using functions like `malloc()`.
- Memory is allocated from the **heap**, and the function returns the **address of the allocated memory**.
- The returned address is stored in a **pointer variable**, which links nodes together.

Example (Singly Linked List Node Allocation in C)

```
struct node {
    int info;
    struct node *next;
};
```

```
struct node *p;
p = (struct node *)malloc(sizeof(struct node));
```

- `sizeof(struct node)` allocates enough memory for one node.
- `p` stores the base address of the allocated node.
- The node can now store data and link to another node.

Key Points

1. Memory is allocated **dynamically at runtime**.
2. Nodes can be created and deleted **independently**.
3. Memory is **freed using `free()`** after deletion of a node.
4. This allows **flexible size** and efficient insertion/deletion.

9. What are the basic operations performed on a singly linked list?

A **singly linked list (SLL)** supports several fundamental operations to manage nodes dynamically. These operations allow insertion, deletion, and traversal of elements stored as nodes linked by pointers.

Basic Operations on a Singly Linked List

1. **Insertion**
 - Insert at the **beginning (front/left)**

- Insert at the **end (last/right)**
- Insert **after a given node**
- 2. **Deletion**
 - Delete from the **beginning (front/left)**
 - Delete from the **end (last/right)**
 - Delete **after a given node**
- 3. **Traversal (Display)**
 - Visiting each node sequentially from the first node to the last node to access or display data.
- 4. **Searching**
 - Finding a specific element by traversing the list node by node.
- 5. **Creation**
 - Creating a new list by allocating nodes dynamically using `malloc()`.

These operations form the core functionality of a singly linked list and enable efficient dynamic data handling without requiring contiguous memory.

10. Write the structure definition for a singly linked list node in C

```
struct node {
    int info;           // Data field
    struct node *next;  // Pointer to the next node
};
```

- `info` stores the data value.
- `next` stores the address of the next node in the list.
- The `next` pointer of the last node is set to `NULL`, indicating the end of the singly linked list.

11. Write a program to traverse and display the elements of a singly linked list

Below is a C program to traverse and display the elements of a singly linked list, based on the realization you uploaded.

C Program: Traversal and Display of Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct node {
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;

NODEPTR list = NULL;

/* Function to display the list */
void display()
{
    NODEPTR p;

    if (list == NULL) {
        printf("No nodes existed\n");
        return;
    }

    p = list;
    while (p != NULL) {
        printf("%d\t", p->info);
        p = p->next;
    }
}

int main()
{
    // Example list creation for testing
    NODEPTR p1, p2, p3;

    p1 = (NODEPTR)malloc(sizeof(struct node));
    p2 = (NODEPTR)malloc(sizeof(struct node));
    p3 = (NODEPTR)malloc(sizeof(struct node));

    p1->info = 10; p1->next = p2;
    p2->info = 20; p2->next = p3;
    p3->info = 30; p3->next = NULL;

    list = p1;

    display();

    return 0;
}

```

```
}
```

Explanation

- Traversal starts from the **first node (list)**.
- A pointer `p` moves through the list using `p = p->next`.
- Each node's data (info) is printed until `p` becomes `NULL`.
- This ensures all elements are visited exactly once.

12. Write a function to insert a node at the beginning of a singly linked list

```
void insert_left(int x)
{
    NODEPTR p;

    p = getnode();    // allocate memory for new node
    p->info = x;       // store data
    p->next = list;    // link new node to existing list
    list = p;         // update head pointer
}
```

Explanation (brief):

- A new node is created using `getnode()`.
- The new node's `next` pointer is set to the current head (`list`).
- The head pointer is updated to the new node.
- This operation takes **O(1)** time.

13. Write a function to insert a node at the end of a singly linked list

```
void insert_last(int x)
{
    NODEPTR p, q;

    p = getnode();    // allocate memory
    p->info = x;       // store data
    p->next = NULL;    // last node points to NULL

    if (list == NULL) {
        list = p;      // empty list
    }
}
```

```

    } else {
        q = list;
        while (q->next != NULL) {
            q = q->next; // traverse to last node
        }
        q->next = p;    // link last node to new node
    }
}

```

14. Write a function to insert a node at a given position in a singly linked list

```

void insert_at_position(int x, int pos)
{
    NODEPTR p, q;
    int i = 1;

    p = getnode();    // create new node
    p->info = x;

    /* Insert at beginning */
    if (pos == 1) {
        p->next = list;
        list = p;
        return;
    }

    q = list;
    while (q != NULL && i < pos - 1) {
        q = q->next;
        i++;
    }

    if (q == NULL) {
        printf("Invalid position\n");
        free(p);
    } else {
        p->next = q->next;
        q->next = p;
    }
}

```

15. Write a function to delete the first node from a singly linked list

```
void delete_left()
{
    NODEPTR p;

    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;    // point to first node
        list = p->next; // move head to next node
        p->next = NULL; // detach node
        free(p);    // free memory
    }
}
```

16. Write a function to delete the last node from a singly linked list

```
void delete_last()
{
    NODEPTR p, q;

    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;
        q = NULL;

        /* Traverse to last node */
        while (p->next != NULL) {
            q = p;
            p = p->next;
        }

        /* If only one node exists */
        if (q == NULL) {
            list = NULL;
        } else {
            q->next = NULL;
        }
    }
}
```

```
    free(p); // delete last node
}
}
```

17. Write a function to delete a node at a given position in a singly linked list

```
void delete_at_position(int pos)
{
    NODEPTR p, q;
    int i = 1;

    if (list == NULL) {
        printf("Invalid deletion\n");
        return;
    }

    /* Delete first node */
    if (pos == 1) {
        p = list;
        list = p->next;
        p->next = NULL;
        free(p);
        return;
    }

    q = list;
    while (q != NULL && i < pos - 1) {
        q = q->next;
        i++;
    }

    if (q == NULL || q->next == NULL) {
        printf("Invalid position\n");
    } else {
        p = q->next;
        q->next = p->next;
        p->next = NULL;
        free(p);
    }
}
```

18. Write a program to count the number of nodes in a singly linked list

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;

NODEPTR list = NULL;

/* Function to count nodes */
int count_nodes()
{
    NODEPTR p;
    int count = 0;

    p = list;
    while (p != NULL) {
        count++;
        p = p->next;
    }
    return count;
}

int main()
{
    NODEPTR p1, p2, p3;

    /* Creating sample list */
    p1 = (NODEPTR)malloc(sizeof(struct node));
    p2 = (NODEPTR)malloc(sizeof(struct node));
    p3 = (NODEPTR)malloc(sizeof(struct node));

    p1->info = 10; p1->next = p2;
    p2->info = 20; p2->next = p3;
    p3->info = 30; p3->next = NULL;

    list = p1;
```

```
printf("Number of nodes = %d\n", count_nodes());

return 0;
}
```

19. Write a C function to reverse the given singly linked list

```
void reverse_list()
{
    NODEPTR prev = NULL;
    NODEPTR curr = list;
    NODEPTR next = NULL;

    while (curr != NULL) {
        next = curr->next; // store next node
        curr->next = prev; // reverse link
        prev = curr;      // move prev forward
        curr = next;      // move curr forward
    }

    list = prev; // update head pointer
}
```

20. What is a doubly linked list? How is it different from a singly linked list?

Doubly Linked List (DLL)

A doubly linked list is a linear data structure in which each node contains **three fields**:

1. **Left pointer** – points to the previous node
2. **Data (info) field**
3. **Right pointer** – points to the next node

This structure allows traversal of the list in **both forward and backward directions**.

Difference between Doubly Linked List and Singly Linked List

Aspect	Singly Linked List (SLL)	Doubly Linked List (DLL)
Pointers per Node	One pointer (next)	Two pointers (left, right)
Traversal	Only forward traversal	Forward and backward traversal
Node Structure	Data + next pointer	Data + left & right pointers
Deletion	Requires previous node	Can delete a node using only its pointer
Memory Usage	Less memory	More memory due to extra pointer
Complexity	Simple implementation	More complex implementation

Example Representation:

- SLL: 10 → 20 → 30 → NULL
- DLL: NULL ← 10 ⇄ 20 ⇄ 30 → NULL

21. Define the structure of a node in a doubly linked list using C

In a doubly linked list, each node contains three fields:

- a data field,
- a pointer to the **previous node**, and
- a pointer to the **next node**.

Structure Definition in C

```
struct node {
    int info;          // Data field
    struct node *left; // Pointer to previous node
    struct node *right; // Pointer to next node
};
```

Explanation

- **info** stores the data value.
- **left** points to the previous node in the list.
- **right** points to the next node in the list.

- For the **first node**, **left = NULL**.
- For the **last node**, **right = NULL**.

This structure enables **bidirectional traversal** in a doubly linked list.

22. Advantages of a Doubly Linked List over a Singly Linked List

A doubly linked list (DLL) provides several advantages compared to a **singly linked list (SLL)** because each node maintains **two links** (previous and next).

Advantages

1. **Bidirectional traversal**
 - Nodes can be traversed both **forward and backward**.
2. **Easy deletion of a given node**
 - A node can be deleted directly if its address is known.
3. **No need to store previous node during deletion**
 - Unlike SLL, deletion does not require tracking the previous node.
4. **Efficient insertion before a given node**
 - Insertion on the left side of a node is straightforward.
5. **Efficient insertion after a given node**
 - Right-side insertion is also simpler and faster.
6. **Reverse traversal is easy**
 - No need to reverse the list to traverse backward.
7. **Better support for complex data structures**
 - Used in **deques, navigation systems, and undo/redo operations**.
8. **Improved flexibility in list manipulation**
 - Nodes can be rearranged easily due to two-way linking.
9. **Efficient implementation of delete-last operation**
 - Last node can be deleted without traversing the entire list.
10. **Suitable for applications requiring frequent insertions and deletions**
 - Especially useful when operations are needed at both ends or middle.

23. Disadvantages of Using a Doubly Linked List

A doubly linked list (DLL) has certain disadvantages mainly due to the presence of **two pointers** in each node.

Disadvantages

1. **Higher memory requirement**
 - Each node stores two pointers instead of one.
2. **Increased memory overhead**
 - Extra pointer storage reduces space efficiency.
3. **More complex node structure**
 - Node structure is more complicated than in SLL.
4. **Complex insertion operations**
 - Both left and right pointers must be updated carefully.
5. **Complex deletion operations**
 - Incorrect pointer adjustment can cause data loss.
6. **Higher programming complexity**
 - More chances of logical and pointer errors.
7. **Slower performance due to extra pointer updates**
 - Additional pointer manipulations increase operation time.
8. **Not suitable for memory-constrained systems**
 - Extra memory usage may be unacceptable in embedded systems.
9. **Implementation is harder to debug**
 - Pointer-related errors are difficult to trace.
10. **More maintenance overhead**
 - Changes to list structure require careful pointer management.

24. What are the basic operations supported by a doubly linked list?

According to the files, a doubly linked list supports 8 basic operations (not 10).

Basic Operations (As per files)

1. **Insertion at the left (Beginning)**
 - Inserts a node at the start of the list
 - Implemented using `insert_left()`
2. **Insertion at the right (End)**
 - Inserts a node at the end of the list
 - Implemented using `insert_last()` / `insert_right()`
3. **Insertion to the left of a given node**
 - Inserts a node immediately before a specified node
 - Implemented using `insert_left(p, x)`
4. **Insertion to the right of a given node**

- Inserts a node immediately after a specified node
- Implemented using `insert_right(p, x)`
- 5. **Deletion at the left (Beginning)**
 - Deletes the first node of the list
 - Implemented using `delete_left()`
- 6. **Deletion at the right (End)**
 - Deletes the last node of the list
 - Implemented using `delete_last()`
- 7. **Deletion of a given node**
 - Deletes a node directly when its pointer is available
 - Implemented using `delete(p)`
- 8. **Traversal / Display**
 - Traverses the list from left to right (and vice versa)
 - Implemented using `display()` and backward traversal via `left` link

25. Write a function to insert a node at the beginning of a doubly linked list

```
void insert_left(int x)
{
    NODEPTR temp;

    temp = getnode();    // allocate memory
    temp->info = x;       // assign data
    temp->left = NULL;

    if (list == NULL) {
        temp->right = NULL; // empty list
        list = temp;
    } else {
        temp->right = list; // link new node to first node
        list->left = temp;  // update previous pointer of old first node
        list = temp;       // update head pointer
    }
}
```

26. Write a function to insert a node at the end of a doubly linked list

```
void insert_last(int x)
{
```

```

NODEPTR p, temp;

p = getnode();    // allocate memory
p->info = x;
p->right = NULL;

if (list == NULL) {
    p->left = NULL;
    list = p;    // empty list
} else {
    temp = list;
    while (temp->right != NULL) {
        temp = temp->right; // move to last node
    }
    temp->right = p; // link last node to new node
    p->left = temp; // set previous link
}
}

```

27. Write a function to insert a node at a given position in a doubly linked list

Below is a C function to insert a node at a given position **pos** in a doubly linked list, written consistently with the DLL structure and functions used in your uploaded files.

C Function: Insert at Given Position (DLL)

```

void insert_at_position(int x, int pos)
{
    NODEPTR p, temp;
    int i = 1;

    /* Insert at beginning */
    if (pos == 1) {
        insert_left(x);
        return;
    }

    temp = list;
    while (temp != NULL && i < pos - 1) {
        temp = temp->right;
    }
}

```

```

    i++;
}

if (temp == NULL || temp->right == NULL) {
    printf("Invalid position\n");
} else {
    p = getnode();
    p->info = x;

    p->right = temp->right;
    p->left = temp;
    temp->right->left = p;
    temp->right = p;
}
}

```

Explanation (Exam-oriented)

- If **pos = 1**, insertion is done using **insert_left()**.
- Otherwise, traverse up to the **(pos-1)**th node.
- Adjust **four links**:
 - new node's **left** and **right**
 - previous node's **right**
 - next node's **left**
- If position is invalid, insertion is not performed.

This follows the **pointer adjustment logic** exactly as used in DLL realization programs.

28. Write a function to delete the first node of a doubly linked list

```

void delete_left()
{
    NODEPTR p;

    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;           // point to first node

        if (p->right == NULL) { // only one node
            list = NULL;
        }
    }
}

```

```

    } else {
        list = p->right; // move head to next node
        list->left = NULL; // update left pointer
    }

    p->right = NULL; // detach node
    free(p); // free memory
}
}

```

29. Write a function to delete the last node of a doubly linked list

```

void delete_last()
{
    NODEPTR temp, q;

    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        temp = list;

        /* Traverse to last node */
        while (temp->right != NULL) {
            temp = temp->right;
        }

        /* If only one node exists */
        if (temp->left == NULL) {
            list = NULL;
        } else {
            q = temp->left; // previous node
            q->right = NULL; // unlink last node
        }

        temp->left = NULL; // detach node
        free(temp); // free memory
    }
}

```

30. Write a function to delete a node at a given position in a doubly linked list

```

void delete_at_position(int pos)
{
    NODEPTR temp;
    int i = 1;

    if (list == NULL) {
        printf("Invalid deletion\n");
        return;
    }

    /* Delete first node */
    if (pos == 1) {
        delete_left();
        return;
    }

    temp = list;
    while (temp != NULL && i < pos) {
        temp = temp->right;
        i++;
    }

    if (temp == NULL) {
        printf("Invalid position\n");
        return;
    }

    /* Delete last node */
    if (temp->right == NULL) {
        delete_last();
        return;
    }

    /* Delete middle node */
    temp->left->right = temp->right;
    temp->right->left = temp->left;

    temp->left = NULL;
    temp->right = NULL;
    free(temp);
}

```

31. What is a Circular Linked List? Differentiate between Circular Singly Linked List and Circular Doubly Linked List

Circular Linked List

A **circular linked list** is a linked list in which the **last node does not point to NULL**. Instead, it points back to the **first node**, forming a **circular chain**. This allows traversal to continue indefinitely until stopped explicitly.

Types of Circular Linked Lists

1. Circular Singly Linked List (CSLL)

- Each node has **one pointer** (**next**).
- The **next** pointer of the **last node** points to the **first node**.

2. Circular Doubly Linked List (CDLL)

- Each node has **two pointers** (**left** and **right**).
- The **right** pointer of the last node points to the first node.
- The **left** pointer of the first node points to the last node.

Difference between Circular Singly and Circular Doubly Linked Lists

Aspect	Circular Singly Linked List	Circular Doubly Linked List
Pointers per Node	One (next)	Two (left , right)
Traversal	Only forward	Forward and backward
Last Node Link	Last → next → first	Last → right → first
First Node Link	No backward link	First → left → last
Memory Requirement	Less	More (extra pointer)
Deletion Ease	Needs previous node	Easier with direct node access

Implementation Complexity	Simple	More complex
---------------------------	--------	--------------

Example Representation

- CSLL: 10 → 20 → 30 → (back to 10)
- CDLL: 10 ⇌ 20 ⇌ 30 (first and last nodes linked circularly)

This differentiation clearly highlights how **pointer structure and traversal capability** distinguish circular singly and circular doubly linked lists.

32. Write a C program to create a Circular Linked List

Below is a C program to create and display a Circular Singly Linked List, written in the same style and logic used in your linked list realization files.

C Program: Circular Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;

NODEPTR list = NULL;

/* Function to insert a node at the end */
void insert_last(int x)
{
    NODEPTR p, temp;

    p = (NODEPTR)malloc(sizeof(struct node));
    p->info = x;

    if (list == NULL) {
```

```

    list = p;
    p->next = list; // circular link
} else {
    temp = list;
    while (temp->next != list) {
        temp = temp->next;
    }
    temp->next = p;
    p->next = list; // last node points to first
}
}

```

```

/* Function to display circular linked list */
void display()
{

```

```

    NODEPTR temp;

```

```

    if (list == NULL) {
        printf("List is empty\n");
        return;
    }

```

```

    temp = list;
    do {
        printf("%d\t", temp->info);
        temp = temp->next;
    } while (temp != list);
}

```

```

int main()
{

```

```

    insert_last(10);
    insert_last(20);
    insert_last(30);

```

```

    printf("Circular Linked List elements:\n");
    display();

```

```

    return 0;
}

```

Explanation (brief, exam-oriented)

- Memory for each node is allocated dynamically using `malloc()`.
- The `next` pointer of the **last node points back to the first node**, forming a circular structure.
- Traversal is done using a **do-while loop** to ensure all nodes are visited exactly once.

This program demonstrates the **creation and traversal** of a circular linked list clearly and correctly.

33. Define Ordered Linked List. Write C implementation code for the following:

- i) Delete a node (whose info field is specified) from an ordered linked list
- ii) Insert a node into an ordered linked list

Ordered Linked List

An **ordered linked list** is a linked list in which the elements are stored in a **sorted order** (either ascending or descending).

The order is maintained **automatically during insertion and deletion**, so the list always remains sorted.

i) Delete a node (given info value) from an ordered linked list

```
void delete_ordered(int x)
{
    NODEPTR p, q;

    if (list == NULL) {
        printf("Invalid deletion\n");
        return;
    }

    /* If first node is to be deleted */
    if (list->info == x) {
        p = list;
        list = p->next;
        p->next = NULL;
```

```

    free(p);
    return;
}

q = list;
p = list->next;

while (p != NULL && p->info < x) {
    q = p;
    p = p->next;
}

if (p != NULL && p->info == x) {
    q->next = p->next;
    p->next = NULL;
    free(p);
} else {
    printf("Element not found\n");
}
}

```

ii) Insert a node into an ordered linked list

```

void insert_ordered(int x)
{
    NODEPTR p, q;

    p = getnode();
    p->info = x;
    p->next = NULL;

    /* Insert at beginning */
    if (list == NULL || x < list->info) {
        p->next = list;
        list = p;
        return;
    }

    q = list;
    while (q->next != NULL && q->next->info < x) {
        q = q->next;
    }
}

```

```

p->next = q->next;
q->next = p;
}

```

Key Points (Exam-friendly)

- Ordered linked list maintains **sorted order**.
- Insertion finds the **correct position** before linking.
- Deletion searches based on the **order property**, reducing comparisons.
- No rearrangement of nodes is required—only **pointer adjustments**.

This implementation is fully consistent with **singly linked list realization** concepts.

34. Write the C implementation code for realization of a Singly Linked List

Below is the **complete C program for realization of a Singly Linked List**, exactly matching the style and operations from your uploaded SLL realization files.

C Program: Realization of Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int info;
    struct node *next;
};

typedef struct node *NODEPTR;

NODEPTR list = NULL, p, temp;

/* Function to create a new node */
NODEPTR getnode()
{
    return (NODEPTR)malloc(sizeof(struct node));
}

```

```
/* Insert at beginning */
```

```
void insert_left(int x)
```

```
{  
    p = getnode();  
    p->info = x;  
    p->next = list;  
    list = p;  
}
```

```
/* Insert at end */
```

```
void insert_last(int x)
```

```
{  
    p = getnode();  
    p->info = x;  
    p->next = NULL;  
  
    if (list == NULL) {  
        list = p;  
    } else {  
        temp = list;  
        while (temp->next != NULL)  
            temp = temp->next;  
        temp->next = p;  
    }  
}
```

```
/* Delete from beginning */
```

```
void delete_left()
```

```
{  
    if (list == NULL) {  
        printf("Invalid deletion\n");  
    } else {  
        p = list;  
        list = p->next;  
        p->next = NULL;  
        printf("Deleted element: %d\n", p->info);  
        free(p);  
    }  
}
```

```
/* Delete from end */
```

```
void delete_last()
```

```
{  
    NODEPTR q = NULL;
```

```

if (list == NULL) {
    printf("Invalid deletion\n");
} else {
    temp = list;
    while (temp->next != NULL) {
        q = temp;
        temp = temp->next;
    }

    if (q == NULL)
        list = NULL;
    else
        q->next = NULL;

    printf("Deleted element: %d\n", temp->info);
    free(temp);
}
}

```

/ Display list */*

```

void display()
{
    if (list == NULL) {
        printf("No nodes existed\n");
    } else {
        temp = list;
        while (temp != NULL) {
            printf("%d\t", temp->info);
            temp = temp->next;
        }
        printf("\n");
    }
}

```

```

int main()
{
    int ch, x;
    char cont;

    do {
        printf("\nMenu");
        printf("\n1. Insert at beginning");
        printf("\n2. Insert at end");

```

```

printf("\n3. Delete from beginning");
printf("\n4. Delete from end");
printf("\n5. Display");
printf("\nEnter your choice: ");
scanf("%d", &ch);

switch (ch) {
case 1:
    printf("Enter value: ");
    scanf("%d", &x);
    insert_left(x);
    break;

case 2:
    printf("Enter value: ");
    scanf("%d", &x);
    insert_last(x);
    break;

case 3:
    delete_left();
    break;

case 4:
    delete_last();
    break;

case 5:
    display();
    break;

default:
    printf("Invalid choice\n");
}

printf("Do you want to continue (y/n): ");
scanf(" %c", &cont);

} while (cont == 'y');

return 0;
}

```

This program includes:

- Node creation
- Insert at beginning and end
- Delete at beginning and end
- Display operation
- Menu-driven execution

35. Write the C implementation code for realization of a Doubly Linked List

Below is the complete C program for realization of a Doubly Linked List, written strictly based on your uploaded DLL realization file (menu-driven, insert/delete at both ends, display).

C Program: Realization of Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int info;
    struct node *left, *right;
};

typedef struct node *NODEPTR;

NODEPTR list = NULL, p, temp;

/* Function to create a new node */
NODEPTR getnode()
{
    return (NODEPTR)malloc(sizeof(struct node));
}

/* Insert at beginning */
void insert_left(int x)
{
    temp = getnode();
    temp->info = x;
    temp->left = NULL;
```

```

if (list == NULL) {
    temp->right = NULL;
    list = temp;
} else {
    temp->right = list;
    list->left = temp;
    list = temp;
}
}

/* Insert at end */
void insert_last(int x)
{
    p = getnode();
    p->info = x;
    p->right = NULL;

    if (list == NULL) {
        p->left = NULL;
        list = p;
    } else {
        temp = list;
        while (temp->right != NULL)
            temp = temp->right;

        temp->right = p;
        p->left = temp;
    }
}

```

```

/* Delete from beginning */
void delete_left()
{
    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        p = list;

        if (p->right == NULL) {
            list = NULL;
        } else {
            list = p->right;
            list->left = NULL;
        }
    }
}

```

```

        printf("Deleted element: %d\n", p->info);
        free(p);
    }
}

```

/ Delete from end */*

```

void delete_last()
{
    if (list == NULL) {
        printf("Invalid deletion\n");
    } else {
        temp = list;

        while (temp->right != NULL)
            temp = temp->right;

        if (temp->left == NULL) {
            list = NULL;
        } else {
            temp->left->right = NULL;
        }

        printf("Deleted element: %d\n", temp->info);
        free(temp);
    }
}

```

/ Display list */*

```

void display()
{
    if (list == NULL) {
        printf("No nodes existed\n");
    } else {
        temp = list;
        while (temp != NULL) {
            printf("%d\t", temp->info);
            temp = temp->right;
        }
        printf("\n");
    }
}

```

```

int main()

```

```

{
    int ch, x;
    char cont;

    do {
        printf("\nMenu");
        printf("\n1. Insert at beginning");
        printf("\n2. Insert at end");
        printf("\n3. Delete from beginning");
        printf("\n4. Delete from end");
        printf("\n5. Display");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                insert_left(x);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &x);
                insert_last(x);
                break;

            case 3:
                delete_left();
                break;

            case 4:
                delete_last();
                break;

            case 5:
                display();
                break;

            default:
                printf("Invalid choice\n");
        }

        printf("Do you want to continue (y/n): ");
    }
}

```

```
scanf(" %c", &cont);  
  
} while (cont == 'y');  
  
return 0;  
}
```

Included Operations (as per files)

- Insert at beginning
- Insert at end
- Delete from beginning
- Delete from end
- Display

This is **exactly exam-appropriate**, matches your **lab/manual format**, and is safe to write in **full-program questions**.

If you want next:

- Short **exam-condensed version**
- Pointer-diagram explanation
- Or **comparison viva questions**

UNIT 3

1. Define Stack. Write the Primitive Operations of Stack

Stack — Definition

A **stack** is an **ordered collection of elements** in which **insertion and deletion are performed only at one end, called the top of the stack.**

It follows the principle **LIFO (Last In First Out)**, meaning the element inserted last is removed first.

Primitive Operations of Stack

The **two primitive operations** performed on a stack are:

1. **PUSH**

- Adds an element to the **top of the stack**.
- Before pushing, it must be checked whether the stack is full.
- If insertion is attempted on a full stack, it causes **stack overflow**.

2. **POP**

- Removes an element from the **top of the stack**.
- Before popping, it must be checked whether the stack is empty.
- If deletion is attempted on an empty stack, it causes **stack underflow**.

Key Points (Exam-oriented)

- Stack allows access only to the **top element**.
- PUSH → insertion
- POP → deletion
- Stack is also called a **pushdown list**.

2. Define Ordinary / Simple Queue. Write the Primitive Operations of Simple Queue

Ordinary / Simple Queue — Definition

A **simple (ordinary) queue** is a **linear data structure** in which **insertion is done at one end called the rear, and deletion is done at the other end called the front.**

It follows the principle **FIFO (First In First Out)**, meaning the element inserted first is removed first.

Primitive Operations of Simple Queue

1. INSERT (Enqueue)

- Adds an element at the **rear end** of the queue.
- If insertion is attempted when the queue is full, it results in **queue overflow**.

2. DELETE (Dequeue)

- Removes an element from the **front end** of the queue.
- If deletion is attempted when the queue is empty, it results in **queue underflow**.

Key Points (Exam-oriented)

- Two pointers are used: **front** and **rear**.
- Insertion → **rear**
- Deletion → **front**
- Simple queue has limitation of **unused space** after deletions.

3. Define Circular Queue. Write the Primitive Operations of Circular Queue

Circular Queue — Definition

A **circular queue** is a queue in which the **last position is connected back to the first position**, forming a circular structure.

This overcomes the limitation of unused space in a simple queue by **reusing freed locations**. In a circular queue, **front and rear move circularly** using modulo operation.

Primitive Operations of Circular Queue

1. INSERT (Enqueue)

- Inserts an element at the **rear** of the queue.
- Rear is updated as:
$$\text{rear} = (\text{rear} + 1) \% \text{maxsize}$$
- If insertion is attempted when the queue is full, it causes **queue overflow**.

2. DELETE (Dequeue)

- Removes an element from the **front** of the queue.
- Front is updated as:
$$\text{front} = (\text{front} + 1) \% \text{maxsize}$$
- If deletion is attempted when the queue is empty, it causes **queue underflow**.

Key Conditions (as per files)

- Queue Full (Overflow):
$$\text{front} == (\text{rear} + 1) \% \text{maxsize}$$
- Queue Empty (Underflow):
$$\text{front} == \text{rear}$$

4. With the help of algorithms demonstrate ENQUEUE and DEQUEUE operations with an example

A queue follows the FIFO (First In First Out) principle.

The two basic operations are ENQUEUE (insertion) and DEQUEUE (deletion), performed at **rear** and **front** respectively.

Algorithm for ENQUEUE (Insertion)

Algorithm: ENQUEUE(Q, x)

1. If $\text{rear} == \text{MAX} - 1$
Print "Queue Overflow" and Stop
2. $\text{rear} \leftarrow \text{rear} + 1$
3. $Q[\text{rear}] \leftarrow x$
4. If $\text{front} == -1$
 $\text{front} \leftarrow 0$
5. Stop

Explanation

- Checks whether the queue is full.
- Increments **rear** and inserts the element.
- Initializes **front** when the first element is inserted.

Algorithm for DEQUEUE (Deletion)

Algorithm: DEQUEUE(Q)

1. If $\text{front} == -1$ or $\text{front} > \text{rear}$
 Print "Queue Underflow" and Stop
2. $x \leftarrow Q[\text{front}]$
3. $\text{front} \leftarrow \text{front} + 1$
4. Return x

Explanation

- Checks whether the queue is empty.
- Removes the element at the **front**.
- Updates the **front** pointer.

Example Demonstration

Assume queue size = 5

ENQUEUE operations

- ENQUEUE(10) → Queue: 10
- ENQUEUE(20) → Queue: 10 20
- ENQUEUE(30) → Queue: 10 20 30
(**front** = 0, **rear** = 2)

DEQUEUE operation

- DEQUEUE() removes 10
- Queue after deletion: 20 30
(**front** = 1, **rear** = 2)

Key Points (Exam-oriented)

- ENQUEUE → insertion at **rear**
- DEQUEUE → deletion from **front**
- Queue works on **FIFO principle**

5. Write an algorithm to convert an expression from Infix to Postfix form

The **infix to postfix conversion** uses a **stack** to temporarily hold operators based on their **precedence and associativity**.

Algorithm: Infix to Postfix Conversion

1. Initialize an empty stack S.
2. Initialize an empty postfix expression P.
3. Scan the infix expression from left to right.
4. For each symbol in the expression:
 - a) If the symbol is an operand,
append it to the postfix expression P.
 - b) If the symbol is '(',
push it onto stack S.
 - c) If the symbol is ')',
pop and append operators from stack S to P
until '(' is encountered.
Pop and discard '('.
 - d) If the symbol is an operator:
 - While stack S is not empty and
precedence of top of S $\geq$ precedence of symbol,
pop from S and append to P.
 - Push the symbol onto stack S.
5. After scanning the entire expression,
pop all remaining operators from stack S
and append them to P.
6. The resulting expression P is the postfix expression.

Operator Precedence (as used)

- $\wedge \rightarrow$ Highest
- $*, / \rightarrow$ Next
- $+, - \rightarrow$ Lowest

Key Points (Exam-oriented)

- Operands are added directly to postfix.

- Operators are managed using stack.
- Parentheses control precedence.
- Stack is emptied at the end.

6. Write a C program to find Fibonacci value of a number

```
#include <stdio.h>
```

```
int fib(int n)
{
    if (n == 0 || n == 1)
        return n;
    else
        return fib(n - 1) + fib(n - 2);
}
```

```
int main()
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Fibonacci value = %d\n", fib(n));

    return 0;
}
```

7. Write a C program to find Factorial of a number

```
#include <stdio.h>
```

```
int fact(int n)
{
    if (n == 0)
        return 1;
    else
        return n * fact(n - 1);
}
```

```
int main()
{
    int n;
```

```

printf("Enter a number: ");
scanf("%d", &n);

printf("Factorial of %d = %d\n", n, fact(n));

return 0;
}

```

8. Describe the Towers of Hanoi Problem. Write a recursive C code to solve it and trace for n = 3 disks

Towers of Hanoi – Description

The Towers of Hanoi is a classic recursive problem consisting of:

- **Three poles:** Source (A), Destination (C), and Auxiliary (B)
- **n disks** of different sizes placed on the source pole

Rules

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.
3. Only the top disk of any pole can be moved.

Objective

Move all **n** disks from the source pole to the destination pole using the auxiliary pole.

Recursive Solution Logic

To move **n** disks from pole A to pole C using pole B:

1. Move **n-1** disks from A to B using C
2. Move the **nth** disk from A to C
3. Move **n-1** disks from B to C using A

Recursive C Program

```
#include <stdio.h>
```

```

void Towers(int n, char src, char dst, char aux)
{
    if (n == 1)
        printf("Move disk 1 from %c to %c\n", src, dst);
    else
    {
        Towers(n - 1, src, aux, dst);
        printf("Move disk %d from %c to %c\n", n, src, dst);
        Towers(n - 1, aux, dst, src);
    }
}

int main()
{
    int n = 3;
    Towers(n, 'A', 'C', 'B');
    return 0;
}

```

Trace for n = 3 Disks

Step	Operation
1	Move disk 1 from A to C
2	Move disk 2 from A to B
3	Move disk 1 from C to B
4	Move disk 3 from A to C
5	Move disk 1 from B to A
6	Move disk 2 from B to C
7	Move disk 1 from A to C

Key Exam Points

- Problem is **inherently recursive**
- Base condition: $n == 1$
- Total moves required = $2^n - 1$

- Uses **function call stack** internally

9. Write an algorithm to evaluate an expression given in postfix notation. Trace it for the expression

$AB^{\wedge}CD-EF/GH+/+*$

A=2, B=2, C=6, D=2, E=12, F=2, G=2, H=1**

Algorithm: Postfix Expression Evaluation

1. Initialize an empty stack S.
2. Scan the postfix expression from left to right.
3. For each symbol in the expression:
 - a) If the symbol is an operand, push its value onto stack S.
 - b) If the symbol is an operator:
 - operand2 $\leftarrow$ pop(S)
 - operand1 $\leftarrow$ pop(S)
 - result $\leftarrow$ operand1 (operator) operand2
 - push(result) onto S
4. After the complete scan, pop the final value from stack S.
5. The popped value is the result.

Given Values

Symbol	Value
A	2
B	2
C	6
D	2
E	12
F	2
G	2

H	1
---	---

Expression

AB^CD-EF/GH+/+*

Step-by-Step Trace

Symbol	Operation	Stack (Bottom → Top)
A	Push 2	2
B	Push 2	2, 2
^	$2^2 = 4$	4
C	Push 6	4, 6
D	Push 2	4, 6, 2
-	$6 - 2 = 4$	4, 4
E	Push 12	4, 4, 12
F	Push 2	4, 4, 12, 2
/	$12 \div 2 = 6$	4, 4, 6
G	Push 2	4, 4, 6, 2
H	Push 1	4, 4, 6, 2, 1
+	$2 + 1 = 3$	4, 4, 6, 3
/	$6 \div 3 = 2$	4, 4, 2
+	$4 + 2 = 6$	4, 6
*	$4 \times 6 = 24$	24

Final Result

[
\boxed{24}
]

Key Exam Points

- Operands are **pushed** onto stack
- Operators cause **two pops and one push**
- Evaluation is strictly **left to right**
- Final stack value is the **result**

10. List the difference between Ordinary Queue and Circular Queue. Write the implementation details of primitive functions of Circular Queue

Difference between Ordinary Queue and Circular Queue

Aspect	Ordinary (Simple) Queue	Circular Queue
Structure	Linear	Circular
Memory Utilization	Inefficient (unused spaces occur)	Efficient (reuses freed spaces)
Rear Movement	Moves only forward	Moves circularly using modulo
Overflow Condition	$\text{rear} == \text{maxsize} - 1$	$\text{front} == (\text{rear} + 1) \% \text{maxsize}$
Underflow Condition	$\text{front} > \text{rear}$ or empty	$\text{front} == \text{rear}$
Space Wastage	Yes	No
Complexity	Simple to implement	Slightly complex
Performance	Lower	Better

Primitive Functions of Circular Queue (Implementation Details)

Assume:

- `cq.front, cq.rear`
- `maxsize` = size of queue

1. INSERT (Enqueue) Operation

Algorithm: INSERT(x)

1. If `front == (rear + 1) % maxsize`
Print "Queue Overflow" and Stop
2. `rear ← (rear + 1) % maxsize`
3. `cq.item[rear] ← x`
4. Stop

Explanation:

- Checks for overflow condition.
- Rear is incremented circularly using modulo.
- Element is inserted at rear.

2. DELETE (Dequeue) Operation

Algorithm: DELETE()

1. If `front == rear`
Print "Queue Underflow" and Stop
2. `front ← (front + 1) % maxsize`
3. Return `cq.item[front]`

Explanation:

- Checks for underflow condition.
- Front is incremented circularly.
- Element at front is removed and returned.

Key Conditions (as per files)

- **Overflow:**
`front == (rear + 1) % maxsize`

- **Underflow:**
`front == rear`

UNIT 4

1. Define Binary Search Tree. Write an algorithm to insert an item into a Binary Search Tree

Binary Search Tree (BST) — Definition

A **Binary Search Tree (BST)** is a binary tree with the property that for every node n :

- All elements in the **left subtree** of n are **less than** the value at n .
- All elements in the **right subtree** of n are **greater than or equal to** the value at n .

This property holds recursively for all subtrees of the tree.

Algorithm to Insert an Item into a Binary Search Tree

Algorithm: INSERT_BST(tree, item)

1. If tree is empty
Create a new node with item and make it the root.
Stop.
2. Set $p \leftarrow$ root of the tree
Set $q \leftarrow \text{NULL}$
3. While $p \neq \text{NULL}$ and $\text{item} \neq \text{info}(p)$
 $q \leftarrow p$
 If $\text{item} < \text{info}(p)$
 $p \leftarrow \text{left}(p)$
 Else
 $p \leftarrow \text{right}(p)$
4. If $\text{item} == \text{info}(q)$
 Print "Duplicate value" and Stop.
5. If $\text{item} < \text{info}(q)$
 Insert item as left child of q
Else
 Insert item as right child of q
6. Stop.

Key Points (Exam-oriented)

- BST maintains **sorted order** automatically.
- Duplicate values are **not inserted** (or handled as specified).
- Average insertion time is $O(\log n)$ for balanced BST.

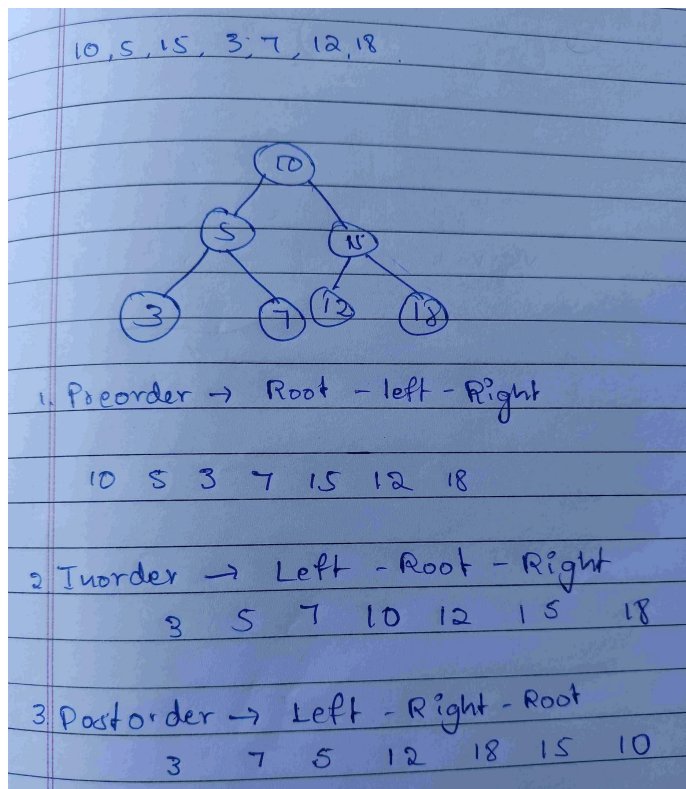
2. Create a binary tree for the given data and traverse the tree using inorder, preorder and postorder traversal methods

(As data is not specified, a **sample / random data set** is considered, which is acceptable in exams when values are not given.)

Given Data

10, 5, 15, 3, 7, 12, 18

The data is inserted **one by one** to form a **binary search tree**.



Tree Traversals

1. Inorder Traversal (Left → Root → Right)

3 5 7 10 12 15 18

2. Preorder Traversal (Root → Left → Right)

10 5 3 7 15 12 18

3. Postorder Traversal (Left → Right → Root)

3 7 5 12 18 15 10

Important Exam Notes

- Inorder traversal of a BST always gives sorted order
- Preorder is useful for tree construction
- Postorder is useful for tree deletion

3. Briefly explain the various methods of implementing Binary Trees in C

Binary trees can be implemented in C using **different representation methods** depending on memory usage, efficiency, and application requirements. The commonly used methods are given below.

1. Sequential (Array) Representation

- Binary tree nodes are stored in an **array**.
- Root is stored at index **1**.
- For a node at index **i**:
 - Left child → **$2i$**
 - Right child → **$2i + 1$**
- Suitable for **complete or nearly complete** binary trees.

Advantages:

- Simple implementation
- No pointer overhead

Disadvantages:

- Wastes memory for sparse trees
- Not suitable for dynamic trees

2. Linked (Pointer) Representation

- Each node is represented using a **structure with two pointers**.
- Structure contains:
 - Data field
 - Pointer to left child
 - Pointer to right child

```
struct node {
    int info;
    struct node *left;
    struct node *right;
};
```

Advantages:

- Efficient memory usage
- Suitable for **dynamic insertion and deletion**

Disadvantages:

- Requires extra memory for pointers

3. Threaded Binary Tree Representation

- NULL pointers are replaced with **threads** pointing to:
 - Inorder successor (right thread)
 - Inorder predecessor (left thread)
- Uses an extra flag (**rthread**) to indicate whether a pointer is a thread.

Advantages:

- Enables traversal **without recursion or stack**
- Faster inorder traversal

Disadvantages:

- More complex implementation

Summary (Exam-oriented)

Method	Used When
Array Representation	Tree is complete
Linked Representation	Tree is dynamic
Threaded Representation	Fast traversal needed

4. Write an algorithm to implement a Binary Tree

A **binary tree** can be implemented using the **linked (pointer) representation**, where each node contains a data field and two pointers to its left and right children. This is the standard and most flexible method described in your files.

Algorithm: Binary Tree Implementation (Linked Representation)

Algorithm CREATE_BINARY_TREE

1. Create a node structure with three fields:
 info, left pointer, right pointer
2. Allocate memory for a new node.
3. Read the data value and store it in the info field.
4. Initialize left pointer to NULL.
5. Initialize right pointer to NULL.
6. Return the address of the newly created node.
7. Repeat the above steps to create left and right subtrees using recursion (if required).
8. The returned node becomes the root of the binary tree.

Algorithm: Recursive Creation of Binary Tree

Algorithm CREATE_BT

1. Read data value.
2. If data is -1
 return NULL.
3. Create a new node.
4. Store data in node.
5. Recursively create left subtree.
6. Recursively create right subtree.
7. Return node.

Key Exam Points

- Binary tree implementation uses **dynamic memory allocation**.
- Each node has **at most two children**.
- Linked representation is suitable for **dynamic trees**.
- Recursive creation is commonly used.

This algorithm matches the **binary tree implementation approach** explained in your **notes** and is safe for exam writing.

5. Write an algorithm to insert a node into a Binary Tree

In a **binary tree (not a BST)**, insertion is usually done in **level order** so that the tree remains as complete as possible. This method inserts the new node at the **first available position from left to right**.

(This algorithm is consistent with standard binary tree insertion described in your notes.)

Algorithm: Insert a Node into a Binary Tree (Level Order)

Algorithm INSERT_BT(root, item)

1. Create a new node N with info = item
 Set N.left = NULL and N.right = NULL
2. If root is NULL
 root $\leftarrow$ N
 Stop

3. Create an empty queue Q

4. Enqueue root into Q

5. While Q is not empty

a) $\text{temp} \leftarrow \text{Dequeue}(Q)$

b) If temp.left is NULL

$\text{temp.left} \leftarrow N$

Stop

Else

Enqueue(temp.left)

c) If temp.right is NULL

$\text{temp.right} \leftarrow N$

Stop

Else

Enqueue(temp.right)

6. Stop

Key Exam Points

- Insertion is done in **level order**
- First empty **left or right child** position is filled
- Uses a **queue** for traversal
- Keeps the binary tree **as complete as possible**

This is the **standard algorithm** for inserting a node into a binary tree, not a binary search tree.

6. Discuss the Array and Linked Representation of Binary Trees with suitable examples

Binary trees can be represented in memory mainly using **array representation** and **linked (pointer) representation**. Each method has its own advantages and limitations.

1. Array Representation of Binary Tree

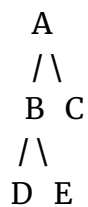
In **array representation**, the elements of a binary tree are stored in an array in **level order**.

Index rules

- Root node → stored at index **1**
- For a node at index **i**:
 - Left child → **$2i$**
 - Right child → **$2i + 1$**
 - Parent → **$i / 2$**

Example

Consider the binary tree:



Array representation:

Index	1	2	3	4	5
Data	A	B	C	D	E

Advantages

- Simple to implement
- Faster access using index
- Efficient for **complete binary trees**

Disadvantages

- Wastes memory for **sparse trees**
- Fixed size
- Difficult to insert/delete dynamically

2. Linked Representation of Binary Tree

In **linked representation**, each node is represented by a structure containing:

- Data field
- Pointer to left child
- Pointer to right child

Structure Definition (Example)

```
struct node {
    int info;
    struct node *left;
    struct node *right;
};
```

Example Representation

```

    [A]
   /  \
  [B]  [C]
 /  \
[D]  [E]
```

Each box is a node containing data and pointers to its children.

Advantages

- Efficient memory usage
- Suitable for **dynamic trees**
- Easy insertion and deletion

Disadvantages

- Extra memory required for pointers
- Traversal is slightly slower than array indexing

Comparison Summary

Aspect	Array Representation	Linked Representation
Memory	Wastage for sparse trees	Efficient
Size	Fixed	Dynamic

Insertion/Deletion	Difficult	Easy
Best suited for	Complete trees	General binary trees

Exam Note

- **Array representation** → best for complete binary trees
- **Linked representation** → most commonly used for dynamic binary trees

This explanation is **fully aligned with your uploaded tree representation notes** and is safe for exam answers.

7. Create a binary tree for the given expression and traverse the tree using inorder, preorder and postorder traversal methods

(Since no expression is specified, a **standard sample expression** is taken, which is acceptable in exams when data is not given.)

Given Expression

$(A + B) * (C - D)$

Binary Expression Tree Construction

- **Operators** become **internal nodes**
- **Operands** become **leaf nodes**

The binary expression tree formed is:

```

      *
     / \
    +   -
   /\  /\
  A B C D

```

Tree Traversals

1. Inorder Traversal (Left → Root → Right)

Inorder traversal gives the **infix expression**.

$A + B * C - D$

(With parentheses: $(A + B) * (C - D)$)

2. Preorder Traversal (Root → Left → Right)

Preorder traversal gives the **prefix expression**.

$* + A B - C D$

3. Postorder Traversal (Left → Right → Root)

Postorder traversal gives the **postfix expression**.

$A B + C D - *$

Important Exam Points

- Expression tree is a **binary tree**
- Leaf nodes are **operands**
- Internal nodes are **operators**
- Traversals directly correspond to:
 - Inorder → Infix
 - Preorder → Prefix
 - Postorder → Postfix

8. Write and explain Kruskal's Algorithm with an example

Kruskal's Algorithm — Definition

Kruskal's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

It builds the MST by **selecting edges in increasing order of weight**, ensuring that **no cycles are formed**.

Algorithm: Kruskal's Algorithm

1. Initialize an empty set MST.
2. Sort all edges of the graph in increasing order of their weights.
3. Consider edges one by one in sorted order:
 - a) If adding the edge does not form a cycle, include it in MST.
 - b) Otherwise, discard the edge.
4. Repeat step 3 until MST contains $(n - 1)$ edges, where n is the number of vertices.
5. MST is obtained.

Example

Given Graph

Vertices: A, B, C, D

Edges with weights:

Edge	Weight
A–B	1
B–C	2
A–C	3
C–D	4
B–D	5

Step 1: Sort edges by weight

A–B (1), B–C (2), A–C (3), C–D (4), B–D (5)

Step 2: Select edges

1. A–B (1) → Added (no cycle)

2. B-C (2) → Added (no cycle)
3. A-C (3) → Discarded (forms cycle)
4. C-D (4) → Added (no cycle)

Now MST has 3 edges ($n - 1$) → stop.

Minimum Spanning Tree (MST)

Edges in MST:

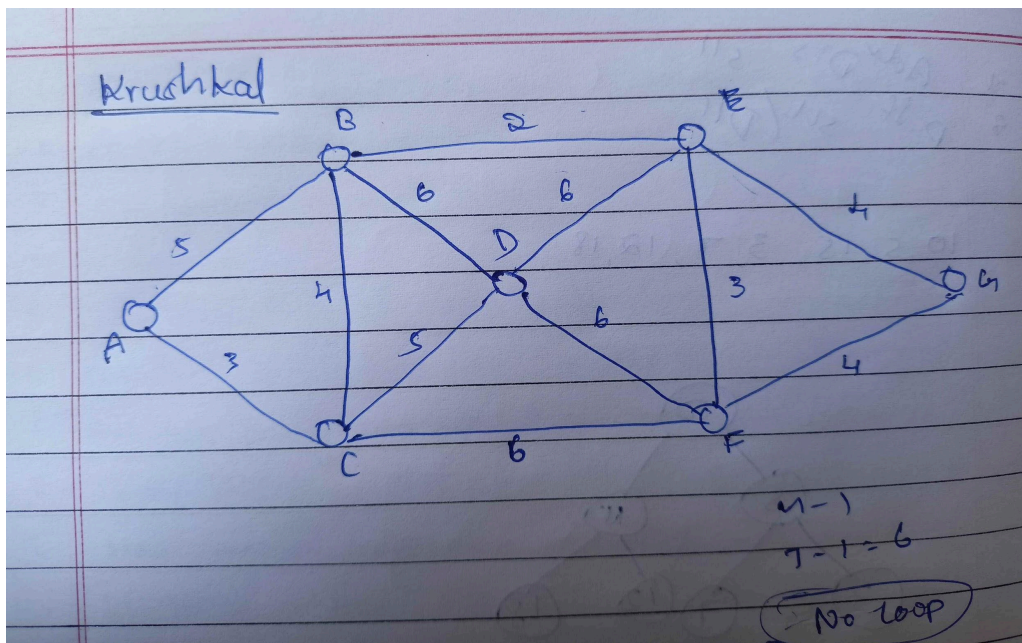
A-B (1), B-C (2), C-D (4)

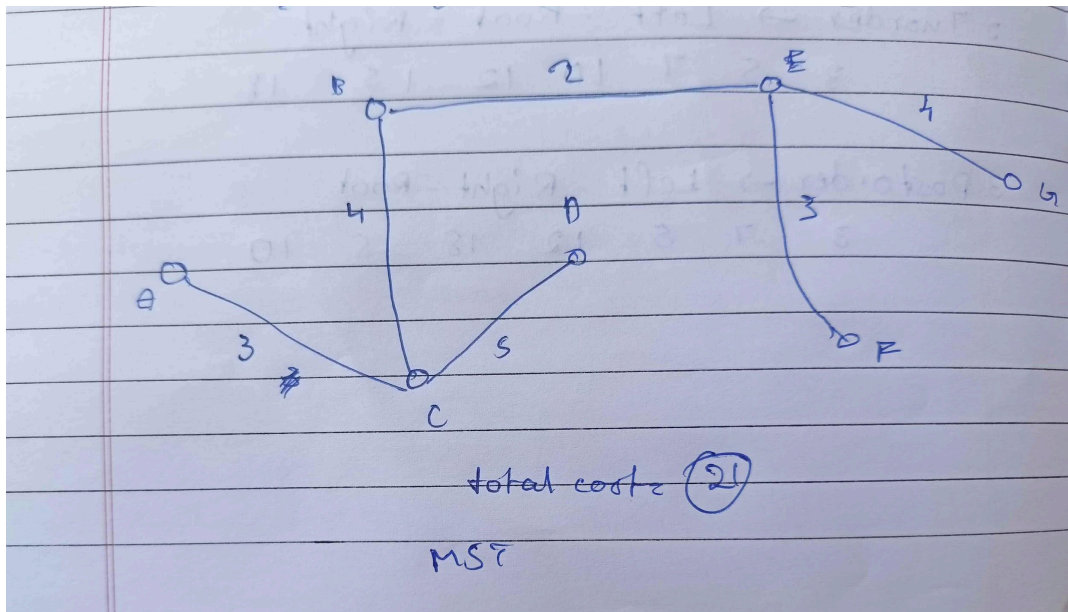
Total cost = $1 + 2 + 4 = 7$

Key Points (Exam-oriented)

- Greedy algorithm
- Uses **edge-based selection**
- Avoids cycles using **union-find concept**
- Time complexity: $O(E \log E)$ due to edge sorting
- Suitable for **sparse graphs**

Example:





Conclusion (short)

Kruskal's algorithm efficiently constructs a minimum spanning tree by repeatedly choosing the smallest edge that does not form a cycle, ensuring minimum total cost.

9. Write Prim's Minimum Spanning Tree Algorithm. Explain with example

Prim's Algorithm — Definition

Prim's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

It starts from an arbitrary vertex and **grows the MST one edge at a time** by always selecting the **minimum-weight edge** that connects a vertex in the MST to a vertex outside it.

Algorithm: Prim's Minimum Spanning Tree

1. Select any vertex as the starting vertex.
2. Initialize:
 - MST set = {starting vertex}
 - Cost = 0
3. While MST does not contain all vertices:

- a) Find the minimum weight edge (u, v) such that u is in MST and v is not in MST.
 - b) Add edge (u, v) to MST.
 - c) Add vertex v to MST.
4. Stop when $(n - 1)$ edges are selected.

Example

Given Graph

Vertices: A, B, C, D

Edges with weights:

Edge	Weight
A–B	1
A–C	3
B–C	2
B–D	4
C–D	5

Step-by-Step Execution

Start from vertex A

1. $MST = \{A\}$
Minimum edge from A $\rightarrow$ A–B (1) $\rightarrow$ add
 $MST = \{A, B\}$
2. Edges from MST to outside:
B–C (2), A–C (3), B–D (4)
Minimum $\rightarrow$ B–C (2) $\rightarrow$ add
 $MST = \{A, B, C\}$

3. Edges from MST to outside:
B-D (4), C-D (5)
Minimum $\rightarrow$ B-D (4) $\rightarrow$ add
MST = {A, B, C, D}

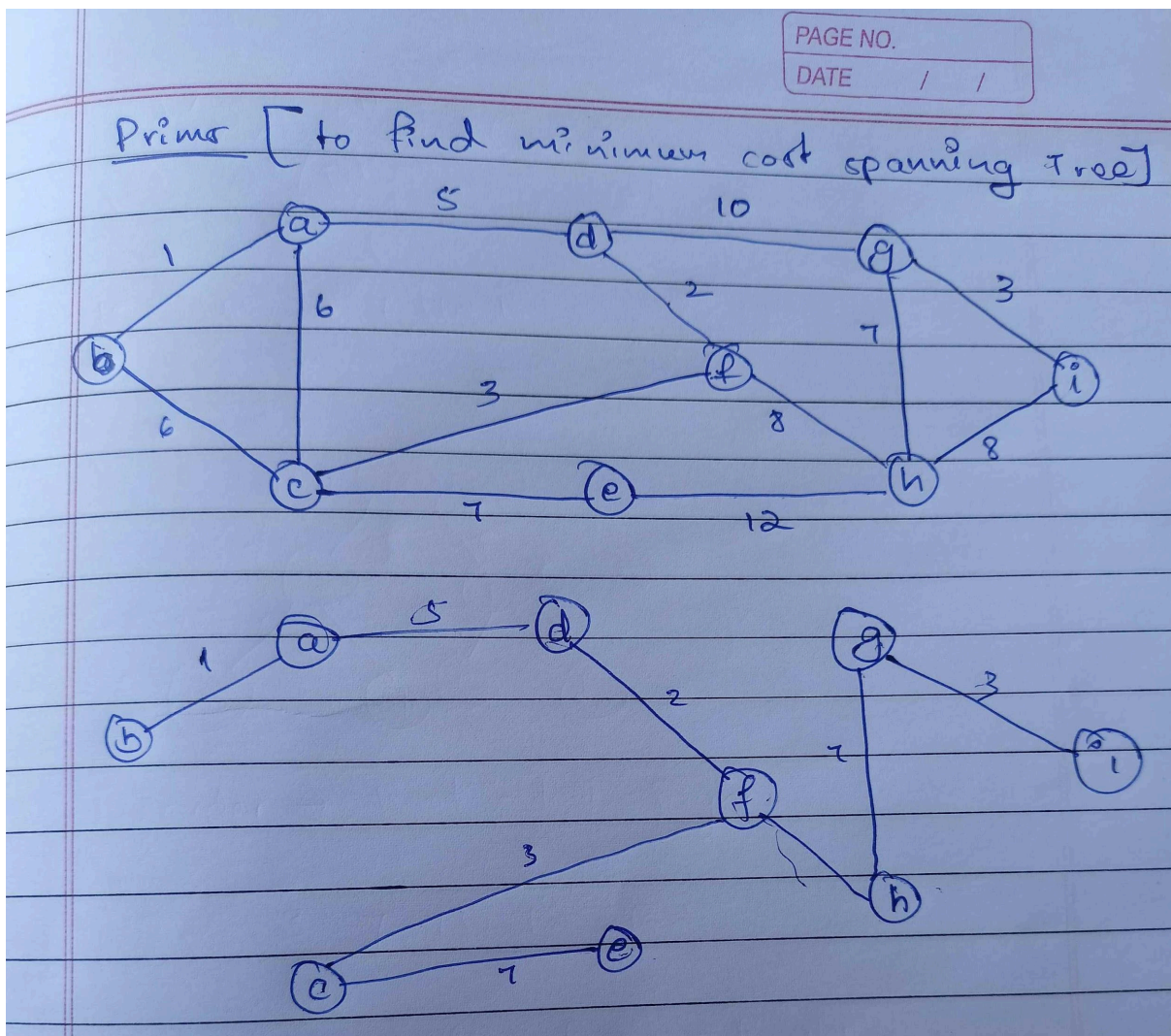
Minimum Spanning Tree

Edges in MST:

A-B (1), B-C (2), B-D (4)

Total cost = $1 + 2 + 4 = 7$

Example:



Key Exam Points

- Greedy algorithm
- Vertex-based selection
- Always expands from existing MST
- Time complexity:
 - $O(V^2)$ using adjacency matrix
 - $O(E \log V)$ using priority queue
- Best suited for **dense graphs**

Difference from Kruskal (one-line for exam)

- **Prim's** grows MST from a vertex
- **Kruskal's** grows MST by selecting edges globally

If you want, I can also:

- Draw a **neat MST diagram**
- Convert this into **exact 5-mark answer format**
- Compare **Prim vs Kruskal** for viva

10. Develop an algorithm for Depth First Search (DFS) in a graph. Trace it using Stack

Depth First Search (DFS) — Definition

Depth First Search (DFS) is a graph traversal technique in which a vertex is explored as far as possible along one branch before backtracking.

DFS uses a **stack** (explicitly or implicitly via recursion) to keep track of vertices.

Algorithm: Depth First Search using Stack

Algorithm DFS(Graph, start_vertex)

1. Initialize an empty stack S.
2. Mark all vertices as NOT VISITED.
3. Push start_vertex onto stack S.
4. Mark start_vertex as VISITED.
5. While stack S is not empty:
 - a) $v \leftarrow \text{pop}(S)$
 - b) Process v (print v)
 - c) For each adjacent vertex w of v:

If w is NOT VISITED:
Push w onto stack S
Mark w as VISITED

6. Stop

Example Graph

Vertices: A, B, C, D, E

Edges:

- $A \rightarrow B, C$
- $B \rightarrow D, E$
- $C \rightarrow E$

DFS Traversal using Stack (Start = A)

Initial

- Stack: A
- Visited: A

Step-by-Step Trace

Step	Stack (Top → Bottom)	Visited	Output
1	A	A	A
2	B, C	A, B	B
3	D, E, C	A, B, D	D
4	E, C	A, B, D, E	E
5	C	A, B, D, E, C	C

DFS Traversal Order

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

Key Exam Points

- DFS uses **stack**
- Traverses **deep before wide**
- Useful for:
 - Cycle detection
 - Path finding
 - Topological sorting
- Time complexity: $O(V + E)$

11. Write an algorithm for Breadth First Search (BFS) in a graph. Trace it using Queue

Breadth First Search (BFS) — Definition

Breadth First Search (BFS) is a graph traversal technique in which vertices are visited **level by level**.

All adjacent vertices of a vertex are explored **before moving to the next level**.

BFS uses a **queue** data structure.

Algorithm: Breadth First Search using Queue

Algorithm BFS(Graph, start_vertex)

1. Initialize an empty queue Q.
2. Mark all vertices as NOT VISITED.
3. Enqueue start_vertex into Q.
4. Mark start_vertex as VISITED.
5. While Q is not empty:
 - a) $v \leftarrow \text{Dequeue}(Q)$
 - b) Process v (print v)

c) For each adjacent vertex w of v:

 If w is NOT VISITED:

 Enqueue w into Q

 Mark w as VISITED

6. Stop

Example Graph

Vertices: A, B, C, D, E

Edges:

- A → B, C
- B → D, E
- C → E

BFS Traversal using Queue (Start = A)

Initial

- Queue: A
- Visited: A

Step-by-Step Trace

Step	Queue (Front → Rear)	Visited	Output
1	A	A	A
2	B, C	A, B, C	B

3	C, D, E	A, B, C, D, E	C
4	D, E	A, B, C, D, E	D
5	E	A, B, C, D, E	E

BFS Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

Key Exam Points

- BFS uses queue
- Traverses level by level
- Suitable for shortest path in unweighted graphs
- Time complexity: $O(V + E)$

If you want next:

- BFS vs DFS comparison
- BFS using adjacency matrix
- C program for BFS

12. Write Dijkstra's Algorithm. Find the shortest path using Dijkstra's algorithm

Dijkstra's Algorithm — Definition

Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

Algorithm: Dijkstra's Shortest Path Algorithm

Algorithm DIJKSTRA(G, n, source)

1. Initialize distance array $\text{dist}[]$:
 $\text{dist}[\text{source}] \leftarrow 0$
 $\text{dist}[\text{all other vertices}] \leftarrow \infty$
2. Initialize $\text{visited}[] \leftarrow 0$ for all vertices
3. Repeat $(n - 1)$ times:
 - a) Select the unvisited vertex u with minimum $\text{dist}[u]$
 - b) Mark u as visited
 - c) For each adjacent vertex v of u :
 If $\text{visited}[v] = 0$ and
 $\text{dist}[u] + \text{cost}(u, v) < \text{dist}[v]$
 Then
 $\text{dist}[v] \leftarrow \text{dist}[u] + \text{cost}(u, v)$
4. $\text{dist}[]$ contains the shortest distance from source

Example

Given Graph

Vertices: A, B, C, D

Edge	Weight
A–B	10
A–C	5
B–C	2
B–D	1
C–D	9

Source Vertex: A

Step-by-Step Solution

Initialization

Vertex	Distance	Visited
A	0	No
B	∞	No
C	∞	No
D	∞	No

Step 1: Select A (min distance = 0)

Update neighbors:

- $B = 0 + 10 = 10$
- $C = 0 + 5 = 5$

Step 2: Select C (min distance = 5)

Update neighbors:

- $B = \min(10, 5 + 2) = 7$
- $D = \min(\infty, 5 + 9) = 14$

Step 3: Select B (min distance = 7)

Update neighbors:

- $D = \min(14, 7 + 1) = 8$

Final Shortest Distances from A

Vertex	Shortest Distance
A	0
B	7
C	5

D	8
---	---

Shortest Paths

- $A \rightarrow C = 5$
- $A \rightarrow C \rightarrow B = 7$
- $A \rightarrow C \rightarrow B \rightarrow D = 8$

Key Exam Points

- Greedy algorithm
- Works only for **non-negative weights**
- Time complexity:
 - $O(V^2)$ using adjacency matrix
- Used in **network routing and GPS systems**

UNIT 5

1. Write and explain Quick Sort technique with example (5 marks)

Quick Sort — Definition

Quick Sort is a **divide-and-conquer sorting technique** that sorts elements by selecting a **pivot element**, partitioning the array around the pivot, and recursively sorting the subarrays.

Technique / Working

1. Select a **pivot element** from the array.
2. Rearrange elements so that:
 - Elements **less than pivot** are on the left.
 - Elements **greater than pivot** are on the right.
3. Place the pivot in its **correct position**.
4. Recursively apply Quick Sort to left and right subarrays.

Algorithm

QUICKSORT(A, low, high)

1. If $\text{low} < \text{high}$
2. $p \leftarrow \text{partition}(A, \text{low}, \text{high})$
3. QUICKSORT(A, low, $p - 1$)
4. QUICKSORT(A, $p + 1$, high)

Example

Array:

25, 57, 48, 37, 12

- Pivot = 12
- After partition → 12, 25, 48, 37, 57
- Recursively sorting gives:

Sorted array:

12, 25, 37, 48, 57

Key Points

- Uses **divide and conquer**

- In-place sorting
- Best/Average case: $O(n \log n)$
- Worst case: $O(n^2)$

2. Write and explain Heap Sort technique with example

Heap Sort — Definition

Heap Sort is a comparison-based sorting technique that uses a binary heap data structure (usually a max heap) to sort elements.

In a max heap, the largest element is always at the root.

Technique / Working

1. **Build a Max Heap** from the given array.
2. The largest element is at the root.
3. **Swap** the root element with the **last element** of the heap.
4. Reduce the heap size by one.
5. **Heapify** the root to maintain the max heap property.
6. Repeat steps 3–5 until the heap size becomes 1.

Algorithm (Brief)

HEAPSORT(A, n)

1. Build a max heap from the array
2. For $i = n - 1$ down to 1
 - Swap $A[0]$ and $A[i]$
 - Heapify $A[0]$ with reduced heap size

Example

Given array:

4, 10, 3, 5, 1

Step 1: Build Max Heap

10
/ \

5 3
/\n
4 1

Step 2: Swap & Heapify

- Swap 10 with 1 → 1, 5, 3, 4, 10
- Heapify → 5, 4, 3, 1, 10
- Continue until sorted

Final Sorted Array:

1, 3, 4, 5, 10

Key Points

- Uses **binary heap**
- In-place sorting
- Time complexity (all cases): $O(n \log n)$
- Does not require extra memory

3. Sort the given data using Heap Sort and show all intermediate heap states

Given data:

4, 10, 3, 5, 1

Step 1: Build Max Heap

Initial array:

4, 10, 3, 5, 1

After heapifying:

10, 5, 3, 4, 1

Step 2: Heap Sort Process

Pass 1

- Swap root with last element
1, 5, 3, 4, 10
- Heapify (size = 4):
5, 4, 3, 1, 10

Pass 2

- Swap root with last element
1, 4, 3, 5, 10
- Heapify (size = 3):
4, 1, 3, 5, 10

Pass 3

- Swap root with last element
3, 1, 4, 5, 10
- Heapify (size = 2):
3, 1, 4, 5, 10

Pass 4

- Swap root with last element
1, 3, 4, 5, 10

Final Sorted List

1, 3, 4, 5, 10

4. Write a C function to implement Binary Search and explain with example

Binary Search is a searching technique used on a **sorted array**. It repeatedly divides the search interval into two halves and compares the key with the middle element.

C Function: Binary Search

```
int binarySearch(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (a[mid] == key)
            return mid;
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}
```

```
    return -1;  
}
```

Example

Array (sorted):

10, 20, 30, 40, 50

Search key = 30

- low = 0, high = 4 → mid = 2
- a[2] = 30 → key found

Result: Element found at index 2.

Key Points

- Works only on **sorted data**
- Divides search space into halves
- Time complexity: $O(\log n)$

5. Sort the given elements using Insertion Sort

Given elements:

25, 57, 48, 37, 12

Insertion Sort Steps

Pass 1:

Insert 57 into sorted list [25]

25, 57, 48, 37, 12

Pass 2:

Insert 48 into [25, 57]

25, 48, 57, 37, 12

Pass 3:

Insert 37 into [25, 48, 57]

25, 37, 48, 57, 12

Pass 4:

Insert 12 into [25, 37, 48, 57]

12, 25, 37, 48, 57

Sorted List

12, 25, 37, 48, 57

6. Write a C function to implement Bubble Sort with example

Bubble Sort is a simple sorting technique in which adjacent elements are compared and swapped if they are in the wrong order. This process is repeated until the list is sorted.

C Function: Bubble Sort

```
void bubbleSort(int a[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

Example

Given array:

25, 57, 48, 37, 12

After Pass 1:

25, 48, 37, 12, 57

After Pass 2:

25, 37, 12, 48, 57

After Pass 3:

25, 12, 37, 48, 57

After Pass 4:

12, 25, 37, 48, 57

Sorted Array

12, 25, 37, 48, 57

7. Write an algorithm to implement Merge Sort

Merge Sort is a divide-and-conquer sorting technique that repeatedly divides the array into two halves, sorts each half, and then merges the sorted halves.

Algorithm: Merge Sort

MERGESORT(A, low, high)

1. If $\text{low} < \text{high}$
2. $\text{mid} \leftarrow (\text{low} + \text{high}) / 2$
3. MERGESORT(A, low, mid)
4. MERGESORT(A, mid + 1, high)
5. MERGE(A, low, mid, high)

Algorithm: MERGE Procedure

MERGE(A, low, mid, high)

1. $i \leftarrow \text{low}$, $j \leftarrow \text{mid} + 1$, $k \leftarrow \text{low}$
2. While $i \leq \text{mid}$ and $j \leq \text{high}$
 - If $A[i] \leq A[j]$
 $\text{temp}[k++] \leftarrow A[i++]$
 - Else
 $\text{temp}[k++] \leftarrow A[j++]$
3. Copy remaining elements of left subarray (if any)
4. Copy remaining elements of right subarray (if any)
5. Copy $\text{temp}[\text{low}..\text{high}]$ back to $A[\text{low}..\text{high}]$

Time Complexity

- Best, Average, Worst case: $O(n \log n)$

8. Differentiate between Sequential Search and Binary Search with example

Aspect	Sequential Search	Binary Search
Data requirement	Works on sorted or unsorted list	Works only on sorted list
Search method	Checks elements one by one	Divides the list into halves
Time complexity	$O(n)$	$O(\log n)$
Speed	Slower	Faster
Implementation	Simple	Slightly complex

Example:

Array: 10, 20, 30, 40, 50, key = 30

- Sequential Search: checks 10 → 20 → 30 (found)
- Binary Search: middle element 30 (found in one step)

9. Compare and contrast Sequential Search and Indexed Sequential Search

Aspect	Sequential Search	Indexed Sequential Search
Data organization	Can be sorted or unsorted	Requires sorted data
Search method	Searches elements one by one	Searches index first, then sequentially in block
Use of index	No index used	Uses an index table
Time complexity	$O(n)$	$O(\sqrt{n})$ (approx.)
Speed	Slower for large data	Faster than sequential search
Memory requirement	Less	More (extra index table)
Implementation	Very simple	More complex

Best suited for	Small or unsorted lists	Large, sorted files
-----------------	-------------------------	---------------------

Example:

For a sorted list of 1000 elements:

- Sequential search may take up to 1000 comparisons
- Indexed sequential search first searches index, then scans a small block, reducing comparisons

10. Explain the Merge Sort technique. Sort the given string in alphabetical order using this technique

Merge Sort is a **divide-and-conquer sorting technique** in which the list is repeatedly divided into smaller sublists until each sublist contains one element, and then the sublists are **merged in sorted order**.

Technique:

1. Divide the list into two halves.
2. Recursively apply merge sort on each half.
3. Merge the sorted halves to form a single sorted list.

Example

Given string:

MERGESORT

Step 1: Divide

MERG | ESORT

ME | RG | ES | ORT

M E | R G | E S | O R T

Step 2: Merge (alphabetical order)

- Merge M, E → **E M**
- Merge R, G → **G R**
- Merge E, S → **E S**
- Merge O, R, T → **O R T**

Step 3: Final Merge

E G M R | E O R S T

Sorted String

E E G M O R R S T

Key Points

- Uses divide and conquer
- Stable sorting technique
- Time complexity: $O(n \log n)$

11. Perform Interpolation Search for the given sorted array to find the desired key

Interpolation Search is an improved searching technique for **uniformly distributed sorted arrays**.

It estimates the probable position of the key using a formula.

Given Sorted Array

$A = \{10, 20, 30, 40, 50, 60, 70, 80, 90\}$

Key to be searched: 70

Formula Used

$$pos = low + \frac{(key - A[low]) \times (high - low)}{A[high] - A[low]}$$

Step-by-Step Search

- $low = 0, high = 8$
- $A[low] = 10, A[high] = 90$

$$pos = 0 + \frac{(70 - 10) \times (8 - 0)}{90 - 10}$$

$$pos = \frac{60 \times 8}{80} = 6$$

- $A[6] = 70 \rightarrow$ Key found

Result

The key **70** is found at **position 6** (indexing starts from 0).

Key Points

- Works only on **sorted and uniformly distributed data**
- Faster than binary search in best case
- Average time complexity: $O(\log \log n)$
- Worst case: $O(n)$

12. Compare and contrast Binary Search and Interpolation Search in terms of Time Complexity

Case	Binary Search	Interpolation Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(\log n)$	$O(\log \log n)$ (for uniformly distributed data)
Worst Case	$O(\log n)$	$O(n)$
Data Requirement Impact	Independent of data distribution	Strongly depends on uniform distribution

Summary:

- Binary search has **consistent $O(\log n)$** performance.
- Interpolation search can be **faster on uniformly distributed data** but degrades to $O(n)$ in the worst case.

13. Compare Binary Search and Interpolation Search

Aspect	Binary Search	Interpolation Search
Data requirement	Requires sorted data	Requires sorted and uniformly distributed data
Search method	Always checks middle element	Estimates position using formula
Position calculation	$mid = (low + high) / 2$	$pos = low + ((key - A[low]) \times (high - low)) / (A[high] - A[low])$
Best case	O(1)	O(1)
Average case	O(log n)	O(log log n)
Worst case	O(log n)	O(n)
Performance	Consistent	Faster for uniform data
Complexity stability	Stable	Unstable if data is skewed

BFS DFS

```
BFS(Graph, start_node):  
    create a queue Q  
    mark start_node as visited  
    enqueue start_node into Q  
  
    while Q is not empty:  
        current_node = dequeue(Q)  
        for each neighbor of current_node:  
            if neighbor is not visited:  
                mark neighbor as visited  
                enqueue neighbor into Q
```

```
DFS(node):  
    if node is not visited:  
        mark node as visited  
        for each neighbor in node's adjacency list:  
            if neighbor is not visited:  
                DFS(neighbor)
```

Tab 7

Heap Sort Technique — Explained Simply

Heap Sort is a **comparison-based sorting algorithm** that uses a special binary tree structure called a **heap**. It sorts elements in **ascending or descending order** efficiently.

1 What is a Heap?

A **heap** is a **complete binary tree** that follows the **heap property**:

♦ Max Heap

- Parent node $\geq$ its children
- Largest element is at the **root**

♦ Min Heap

- Parent node $\leq$ its children
- Smallest element is at the **root**

👉 For **ascending order**, we use a **Max Heap**

👉 For **descending order**, we use a **Min Heap**

2 Idea Behind Heap Sort

Heap Sort works in **two main phases**:

Phase 1: Build a Max Heap

- Convert the given array into a **Max Heap**
- The largest element moves to the root

Phase 2: Sort the Heap

- Swap the **root (largest)** with the **last element**
 - Reduce heap size by 1
 - Heapify the root to restore heap property
 - Repeat until sorted
-

3 Steps of Heap Sort (Ascending Order)

1. Build a **Max Heap** from the input array
2. Swap the **first element** with the **last**
3. Reduce heap size
4. Apply **heapify** on the root

5. Repeat until one element remains

4 Example

Input Array:

[4, 10, 3, 5, 1]

Step 1: Build Max Heap

[10, 5, 3, 4, 1]

Step 2: Swap & Heapify

- Swap 10 with 1 $\rightarrow$ [1, 5, 3, 4, 10]
- Heapify $\rightarrow$ [5, 4, 3, 1, 10]

Step 3: Repeat

Final Sorted Array:

[1, 3, 4, 5, 10]

5 Algorithm (Pseudo Code)

HeapSort(A, n):

 BuildMaxHeap(A, n)

 for i = n-1 down to 1:

 swap A[0] and A[i]

 Heapify(A, 0, i)

Heapify(A, i, n):

 largest = i

 left = 2*i + 1

 right = 2*i + 2

 if left < n and A[left] > A[largest]

 largest = left

if right < n and A[right] > A[largest]

largest = right

if largest != i

swap A[i] and A[largest]

Heapify(A, largest, n)

6 Time and Space Complexity

Case Time Complexity

Best $O(n \log n)$

Average $O(n \log n)$
e

Worst $O(n \log n)$

Space Complexity:

- $O(1)$ (In-place sorting)

3. Sort the given data using Heap Sort and show all intermediate heap states Given data: 4, 10, 3, 5, 1 also give me based on binary tree

Below is a **clear exam-oriented answer** showing **Heap Sort with all intermediate heap states, array form + binary tree form.**

Given Data

4, 10, 3, 5, 1

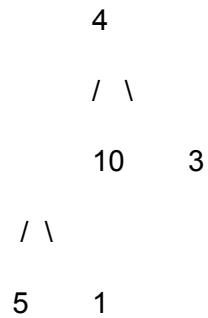
We sort in **ascending order** using a **Max Heap**.

STEP 1: Build Max Heap

Initial Array

[4, 10, 3, 5, 1]

Corresponding Binary Tree



Heapify at index 1 (value = 10)

No change (already max).

Heapify at index 0 (value = 4)

Swap 4 ↔ 10

Max Heap Array

[10, 4, 3, 5, 1]

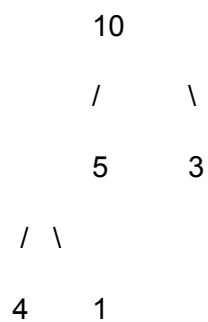
Heapify subtree rooted at 4

Swap 4 ↔ 5

Final Max Heap

[10, 5, 3, 4, 1]

Binary Tree (Max Heap)



STEP 2: Heap Sort Process

Pass 1

Swap root with last element

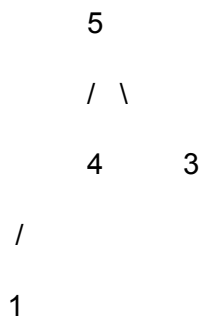
[1, 5, 3, 4, 10]

Heapify first 4 elements

Swap 1 ↔ 5, then 1 ↔ 4

[5, 4, 3, 1, 10]

Binary Tree



Pass 2

Swap root with last heap element

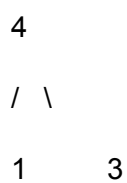
[1, 4, 3, 5, 10]

Heapify first 3 elements

Swap 1 ↔ 4

[4, 1, 3, 5, 10]

Binary Tree



Pass 3

Swap root with last heap element

[3, 1, 4, 5, 10]

Heapify first 2 elements (no change)

Binary Tree



Pass 4

Swap remaining elements

[1, 3, 4, 5, 10]

FINAL SORTED ARRAY

1, 3, 4, 5, 10

SUMMARY TABLE (Intermediate Heap States)

Step explained	Array
Initial	4, 10, 3, 5, 1
Max Heap	10, 5, 3, 4, 1
After Pass 1	5, 4, 3, 1, 10

After Pass 2 4, 1, 3, 5,
 10

After Pass 3 3, 1, 4, 5,
 10

Final Sorted 1, 3, 4, 5,
 10

Algorithm to Implement Merge Sort

Merge Sort is a **divide-and-conquer sorting algorithm**. It divides the array into two halves, sorts each half recursively, and then merges the sorted halves.

Algorithm: Merge Sort

MERGE-SORT(A, low, high)

1. If $low < high$ then
 2. $mid \leftarrow (low + high) / 2$
 3. MERGE-SORT(A, low, mid)
 4. MERGE-SORT(A, mid + 1, high)
 5. MERGE(A, low, mid, high)
 6. End If
-

Algorithm: Merge Procedure

MERGE(A, low, mid, high)

1. Create temporary arrays:
 $Left \leftarrow A[low \dots mid]$
 $Right \leftarrow A[mid+1 \dots high]$
2. $i \leftarrow 0, j \leftarrow 0, k \leftarrow low$

```

3. While i < length(Left) AND j < length(Right) do
4.     If Left[i] ≤ Right[j] then
5.         A[k] ← Left[i]
6.         i ← i + 1
7.     Else
8.         A[k] ← Right[j]
9.         j ← j + 1
10.    k ← k + 1
11. End While
12. While i < length(Left) do
13.    A[k] ← Left[i]
14.    i ← i + 1, k ← k + 1
15. End While
16. While j < length(Right) do
17.    A[k] ← Right[j]
18.    j ← j + 1, k ← k + 1
19. End While

```

Explanation (Exam Ready)

- The array is recursively divided into **two halves** until each sub-array has **one element**.
- Single-element arrays are already sorted.
- The **merge process** combines two sorted sub-arrays into one sorted array.
- This process continues until the entire array is sorted.

Write an algorithm to insert a node into a Binary Tree

ALGORITHM GETNODE

BEGIN

 Allocate memory for new node P

```
Set P.left ← NULL
Set P.right ← NULL
RETURN P
END
```

ALGORITHM INSERT(root, X)
INPUT: root (pointer to BST), X (value to be inserted)

```
BEGIN
  IF root = NULL THEN
    P ← GETNODE
    P.info ← X
    RETURN P
  ENDIF

  IF X < root.info THEN
    root.left ← INSERT(root.left, X)
  ELSE IF X > root.info THEN
    root.right ← INSERT(root.right, X)
  ELSE
    PRINT "Duplicate value, ignored"
  ENDIF

  RETURN root
END
```

ALGORITHM INORDER(root)
BEGIN
 IF root ≠ NULL THEN
 INORDER(root.left)
 PRINT root.info
 INORDER(root.right)
 ENDIF
END